



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

BCSE305L – Embedded Systems

Module 6.1

EMBEDDED NETWORKING PROTOCOLS
I2C, CAN

Module:6 Embedded Networking Protocols 5 hours

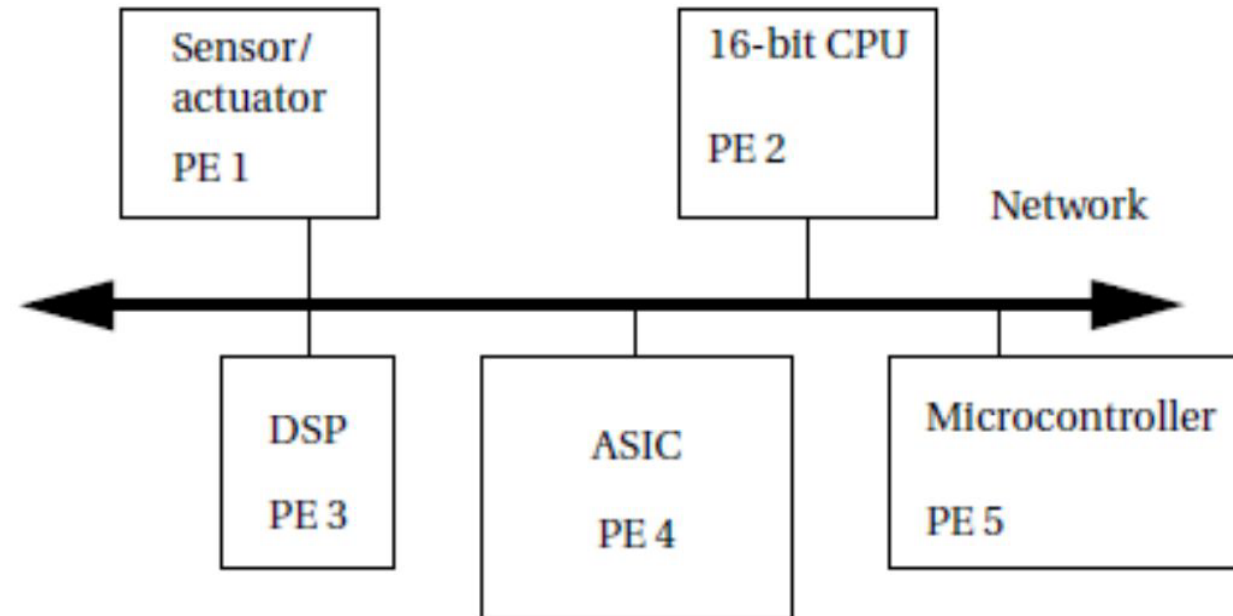
Inter Integrated Circuits (I2C), Controller Area Network, Embedded Ethernet Controller, RS232, Bluetooth, Zigbee, Wifi.

EMBEDDED NETWORKING PROTOCOLS

- Communication Protocols are a **set of rules** that **allow two or more communication systems** to communicate data via any physical medium.
- Embedded System is an electronic system or device which employs both hardware and software.
- A processor or controller takes input from the physical world peripherals like sensors, actuators etc., processes the same through appropriate software and provides the desired output.
- In this case, the **components have to communicate** with each other to provide the anticipated output. Each communicating entity should agree to some **protocol to exchange information**.
- **Many different protocols** are available for embedded systems and are **deployed depending upon the application area**.
- In general, the communication protocols is associated with **physical layer** describing the **signals incorporated, signal strength, hand shaking mechanism, device addressing, wired or wireless, data lines etc.**

Embedded system Architecture

- A PE may be an **instruction set processor** such as a DSP, CPU, or microcontroller, as well as a **nonprogrammable unit** such as the ASICs
- An **I/O device** such as sensor or actuator may also be a PE.
- The connection between PEs provided by the network as a communication link.



Distributed Embedded Systems



© Vector Informatik GmbH

In this car, you can see many small components (shown as colored blocks). These are called ECUs – Electronic Control Units. All these ECUs communicate with each other using protocols like CAN, I²C, etc.

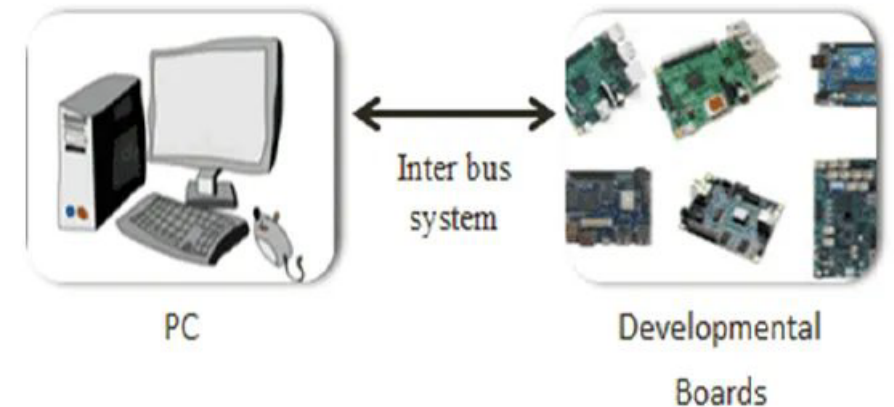
- In a distributed embedded system, **several processing elements (PEs)** (either microprocessors or ASICs) are connected by a **network** that allows them to communicate.
- The application is distributed over the PEs, and some of the work is done at each node in the network.
- An important advantage of a distributed system with several CPUs is that **one part of the system can be used to help diagnose problems in another part.**
- Whether you are debugging a prototype or diagnosing a problem in the field, isolating the error to one part of the system can be difficult when everything is done on a single CPU.
- If you have several CPUs in the system, **you can use one to generate inputs for another and to watch its output.**

EMBEDDED NETWORKING PROTOCOLS

Types of Communication Protocols in Embedded Systems

- **Inter System Protocol:** **Inter system protocols establish communication between two communicating devices** i.e. between PC and microprocessor kit, developmental boards, etc. In this case, the communication is achieved through

- USB Communication protocols
- UART Communication protocols
- USART Communication protocols



USB → Standard interface for connecting devices (like keyboard, mouse, pen drive)

UART → Serial communication without a clock (asynchronous)

USART → Can work both with clock (synchronous) and without clock (asynchronous)

Intra System Protocol: The Intra system protocol establishes **communication between components within the circuit board**. In embedded systems, intra system protocol increases the number of components connected to the controller. Increase in components lead to circuit complexity and increase in power consumption. Intra system protocol promises secure access of data from the peripherals.

- I2C Protocol
- SPI Protocol
- CAN Protocol

- Easy to add new components without redesigning the system

I2C-Inter - Integrated Circuit

Philips invented the Inter-Integrated Circuits protocol (IIC or I2C) in 1982.

The I2C protocol **connects multiple slaves to a single master** or various slaves to multiple masters.

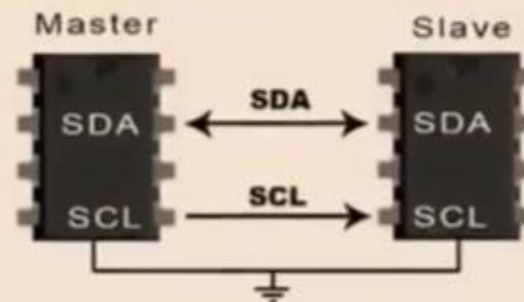
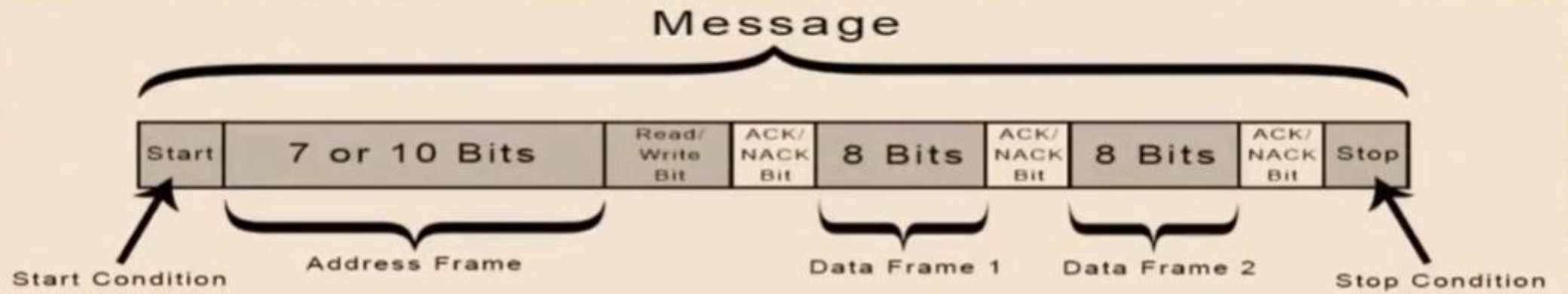
The I2C protocol operates when **two microcontrollers send data to the same memory or send identical data** to the LCD with many other applications.

The I2C serial communication protocol has two signals: **Serial Data (SDA) and Serial Clock (SCL)**. The **master and slave receive data through the SDA line**, and SCL is the clock signal line.

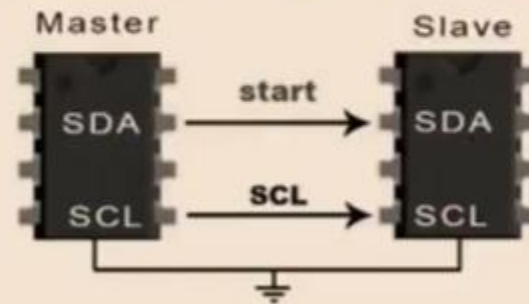
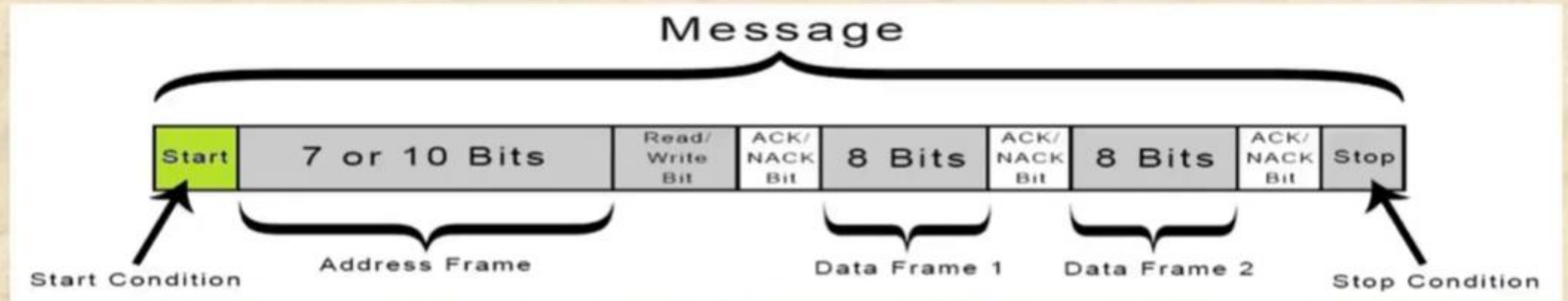
Terms Used:

- **Master** - Device which control the communication
- **Slave** - Device which respond to communication
- **SDA** - Serial Data : Data Transfer line
- **SCL** - Serial Clock : Timing clock line
- **Ack** - To conform byte received successfully
- **Nack** - No received properly / Unable to process
- **Start** - Initiate the data transfer
- **Stop** - Terminate the data transfer
- **R/W** - Read / Write Command Bit
- **Addr** - Address of Devices
- **Data** - Data to be transferred

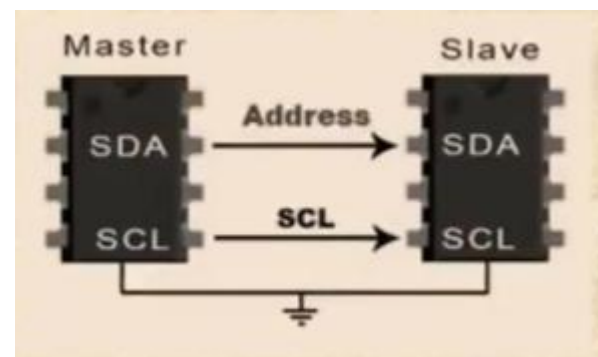
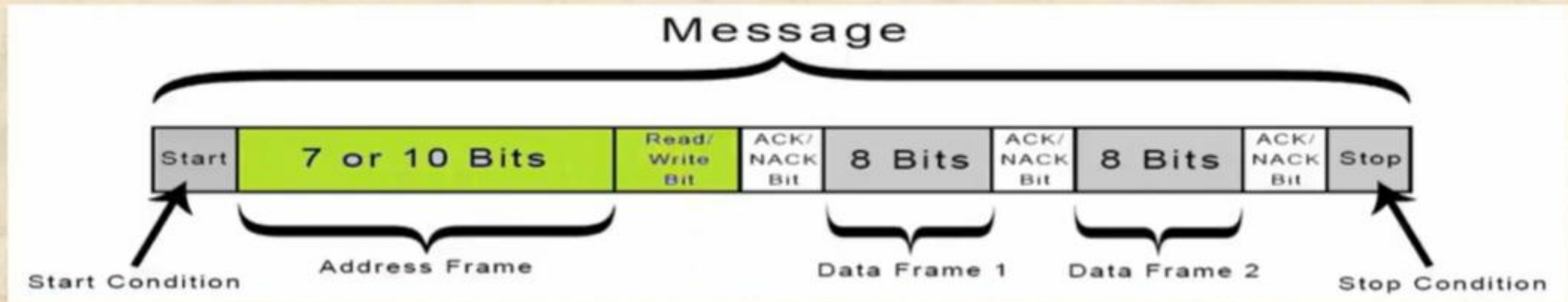
Frame



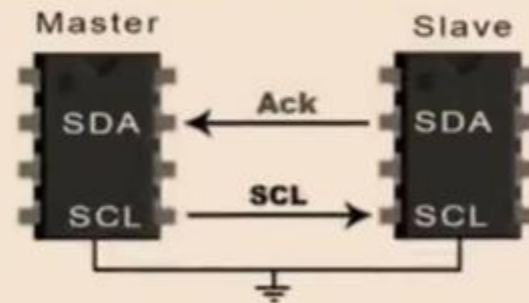
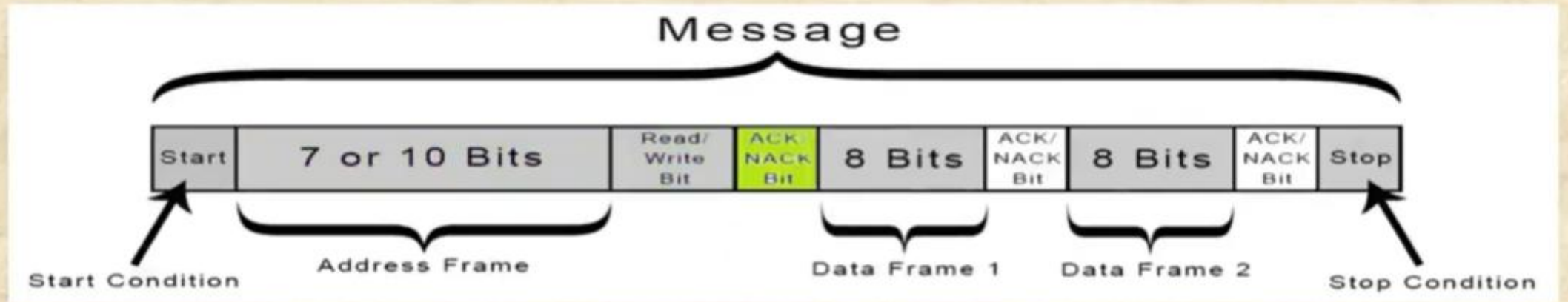
Frame



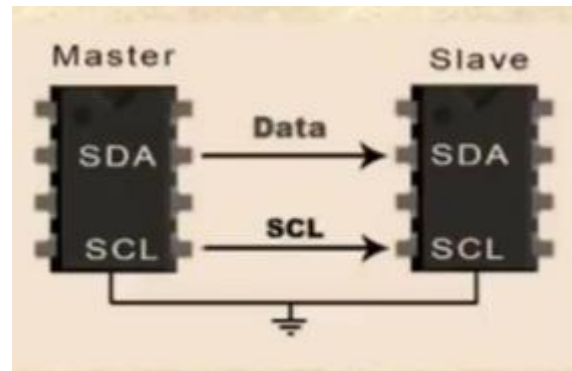
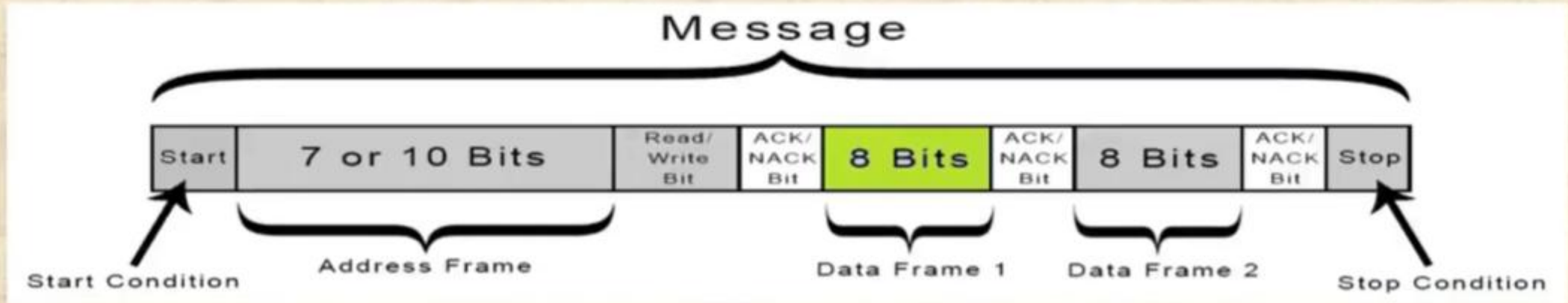
Frame



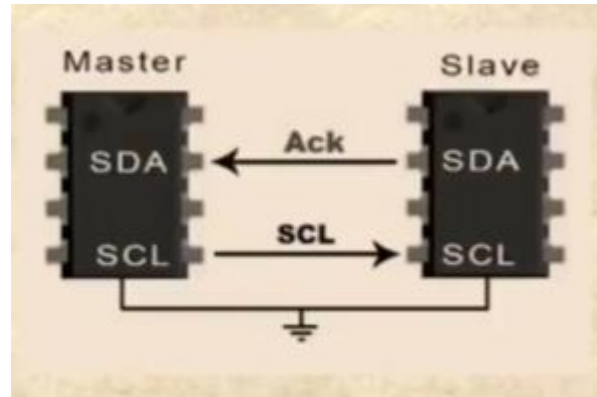
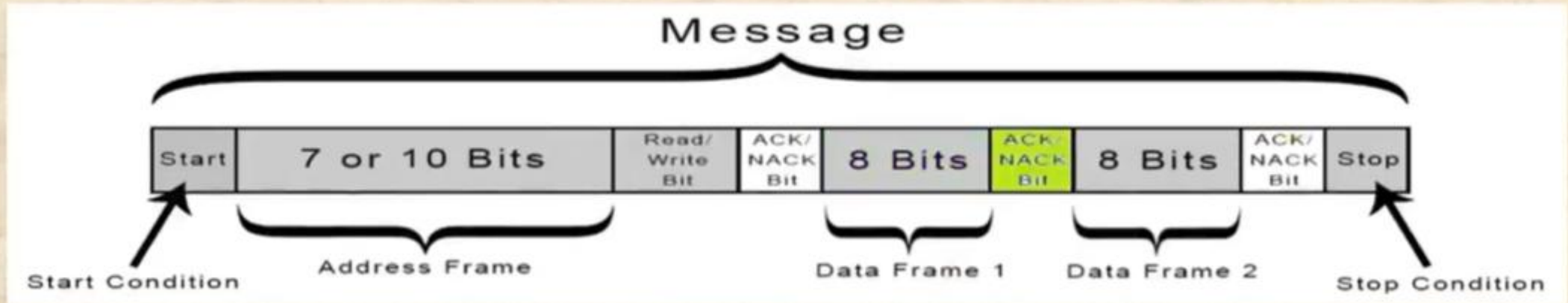
Frame



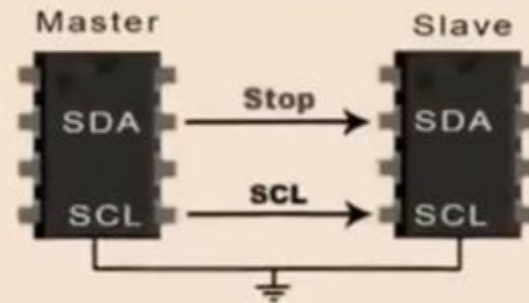
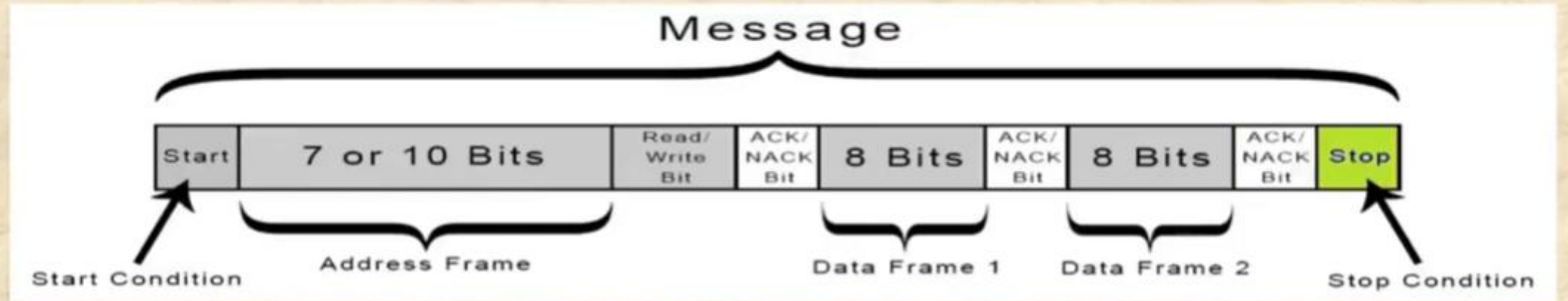
Frame



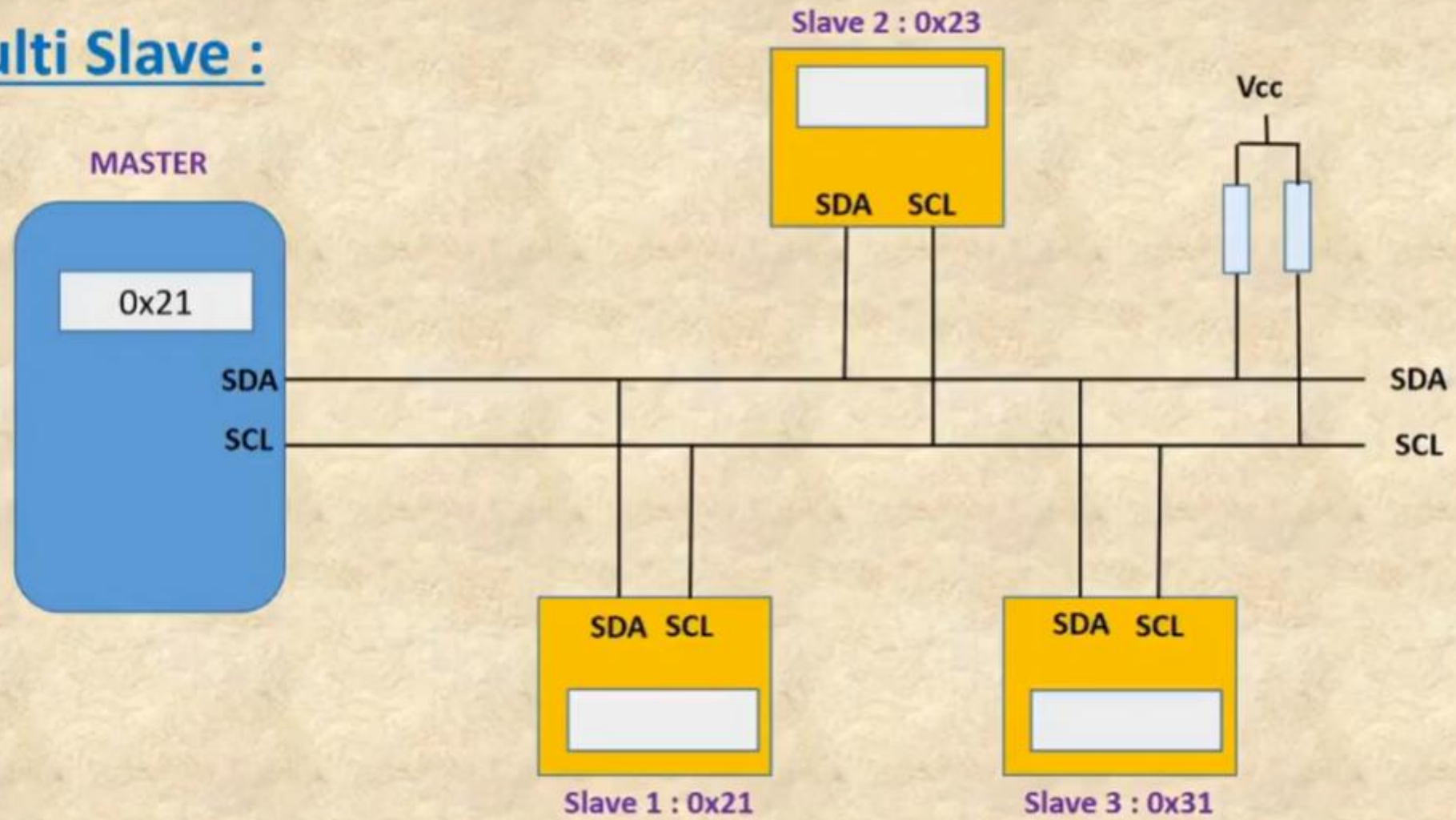
Frame



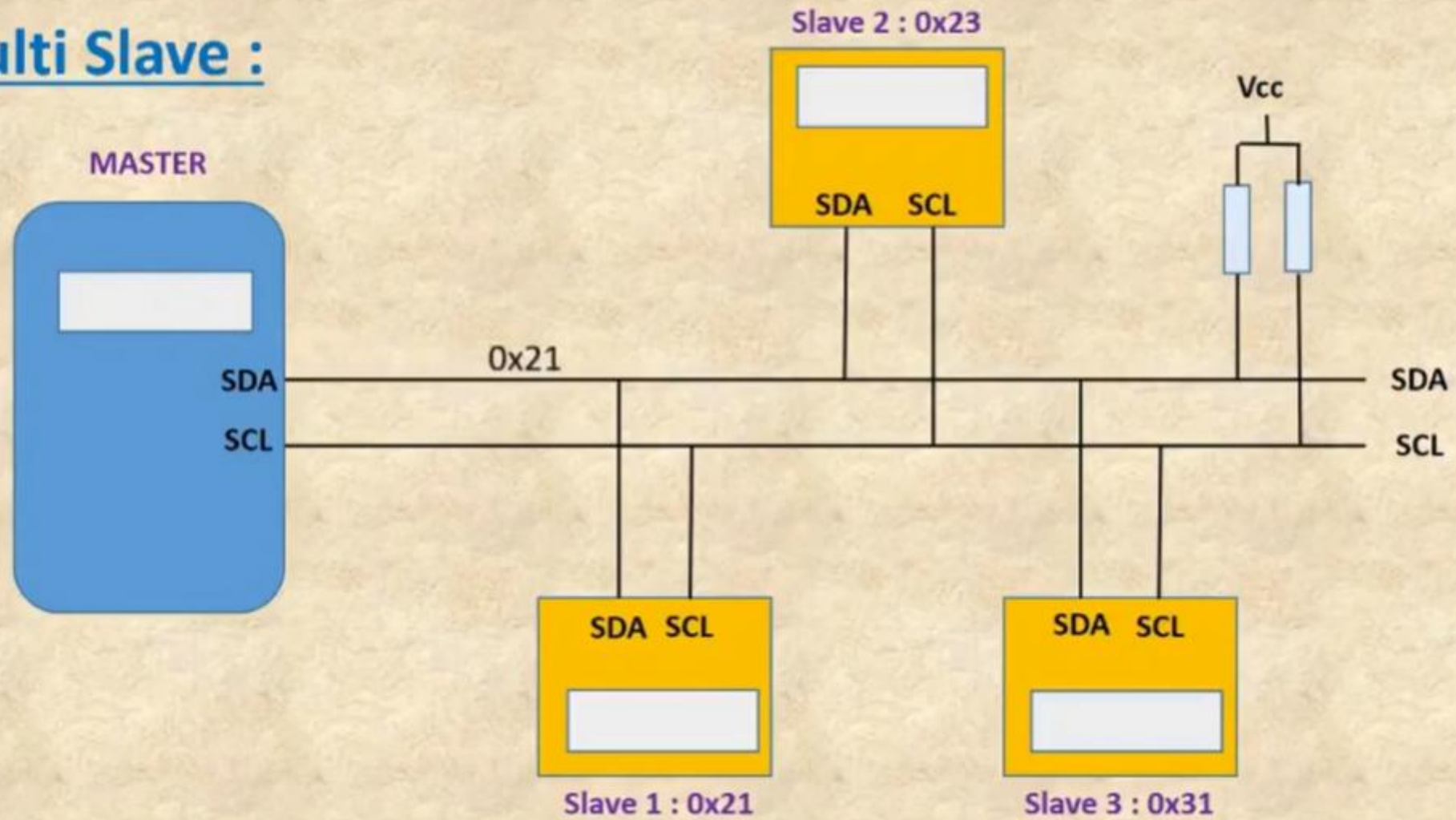
Frame



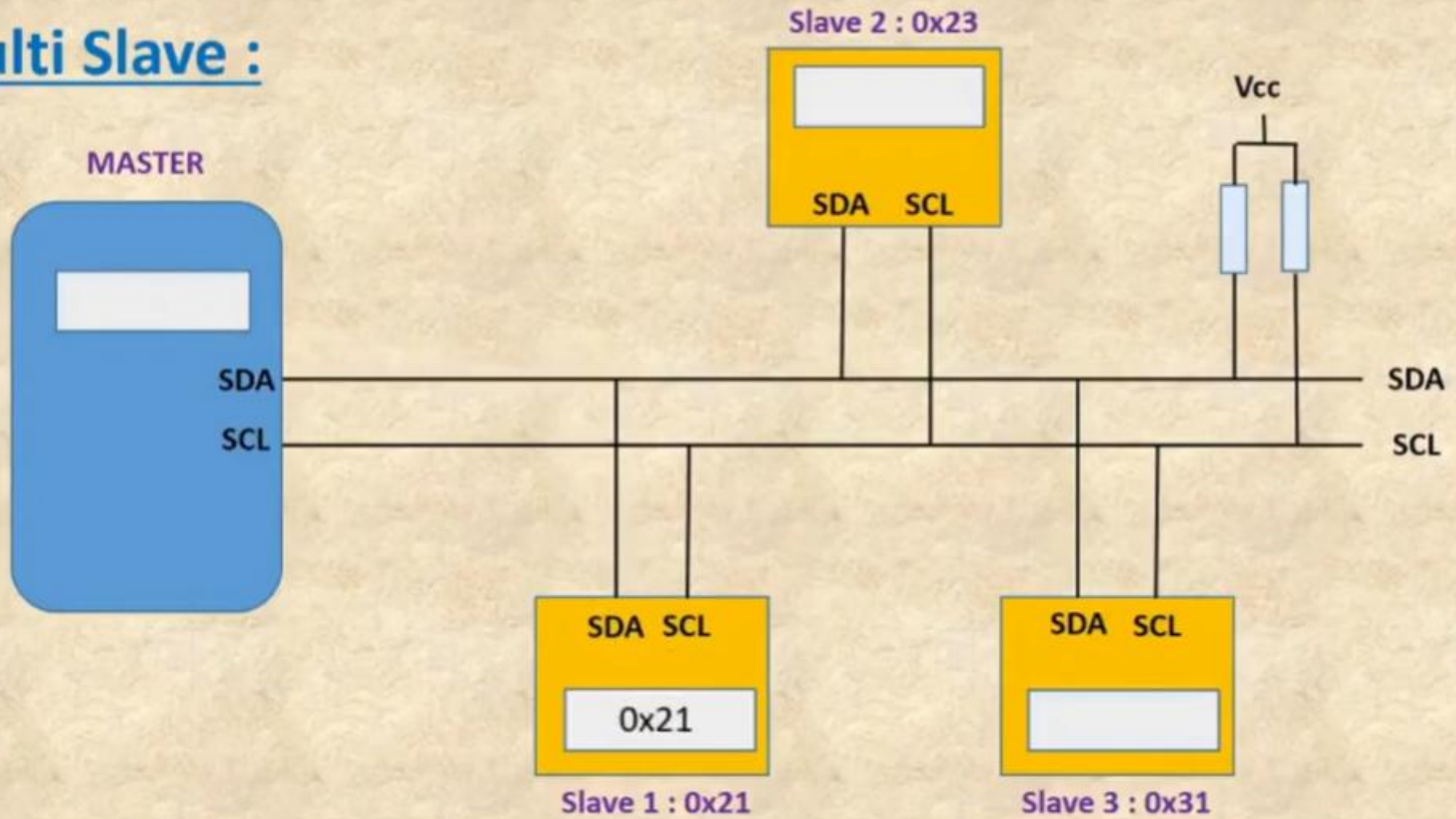
Multi Slave :



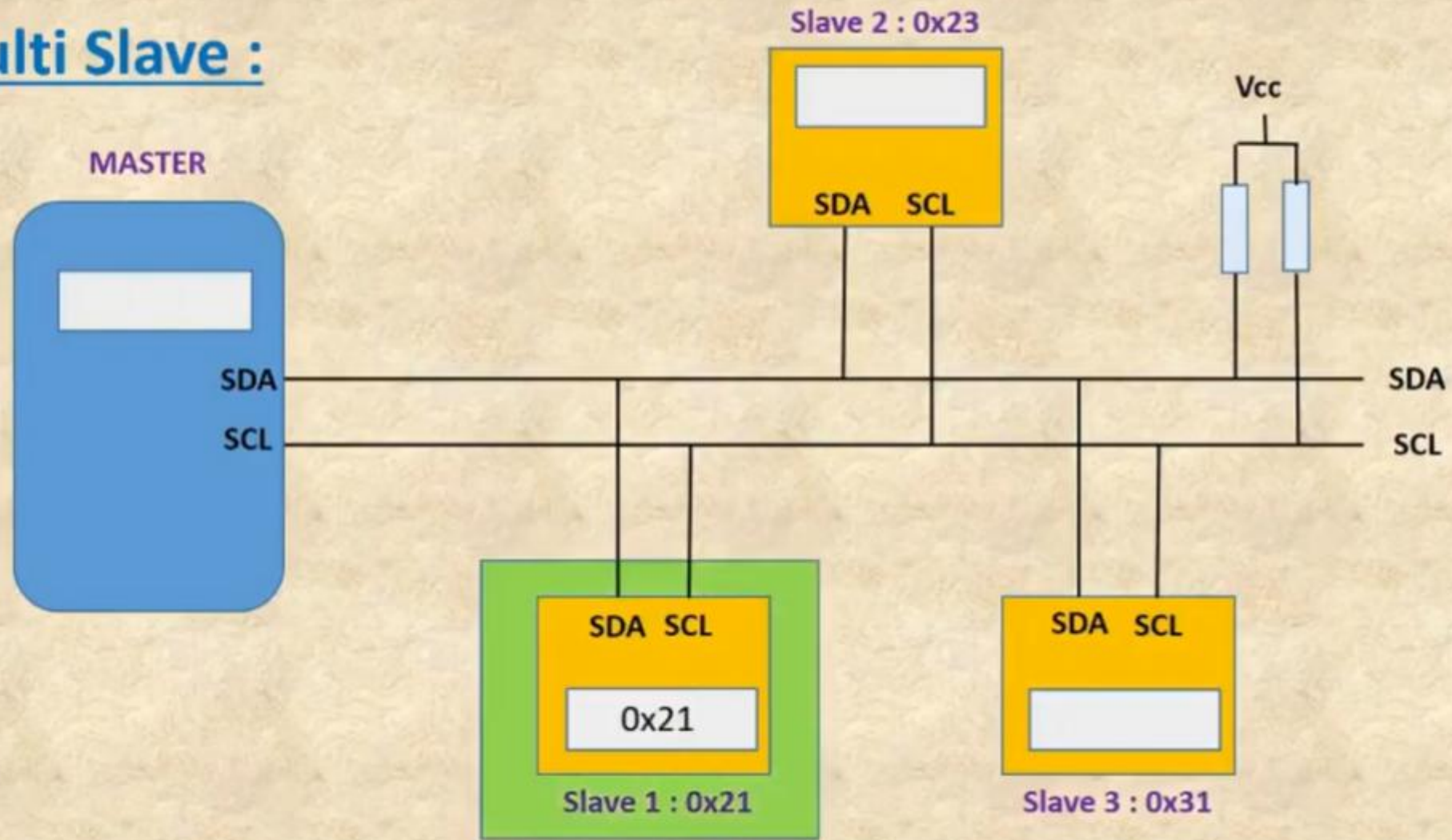
Multi Slave :



Multi Slave :

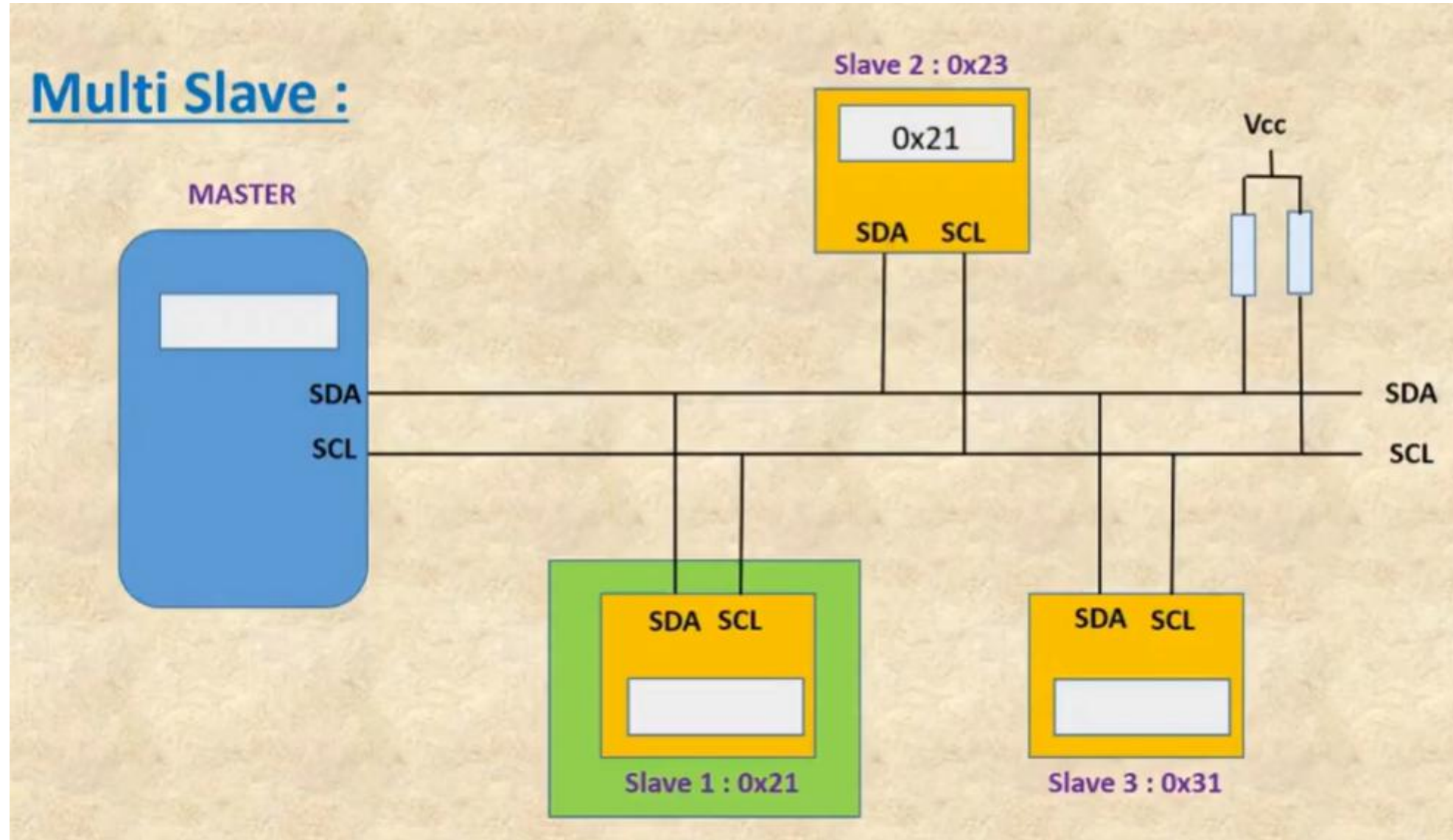


Multi Slave :



This IC is ready to communicate

Address is not matched, ignores transaction



I2C-Inter - Integrated Circuit

- In order to establish communication, **master device initially sends the target slave address along with R/W (Read/Write) flag.**
- The corresponding **slave device will move into active mode** leaving other devices in off state.
- Once the slave device is ready, **communication starts** between master and slave devices.
- **One bit acknowledgment is replied** by the receiver if transmitter transmits 1 byte (8 bits) of data.
- **A stop condition is issued** at the end of communication between devices.

I2C-Inter - Integrated Circuit

The I2C protocol supports, depending upon the modes of operation, different data speeds ranging from **100kbit/s** in **standard mode** to **3.4 Mbps** in **full speed mode**.

I2C protocols use data from hardware sensors, communicate with

- multiple microcontrollers,
- display control of cellphones,
- change color settings of the monitor color,
- assess ADCs and DACs,
- circuits, and
- read specific memory ICs.

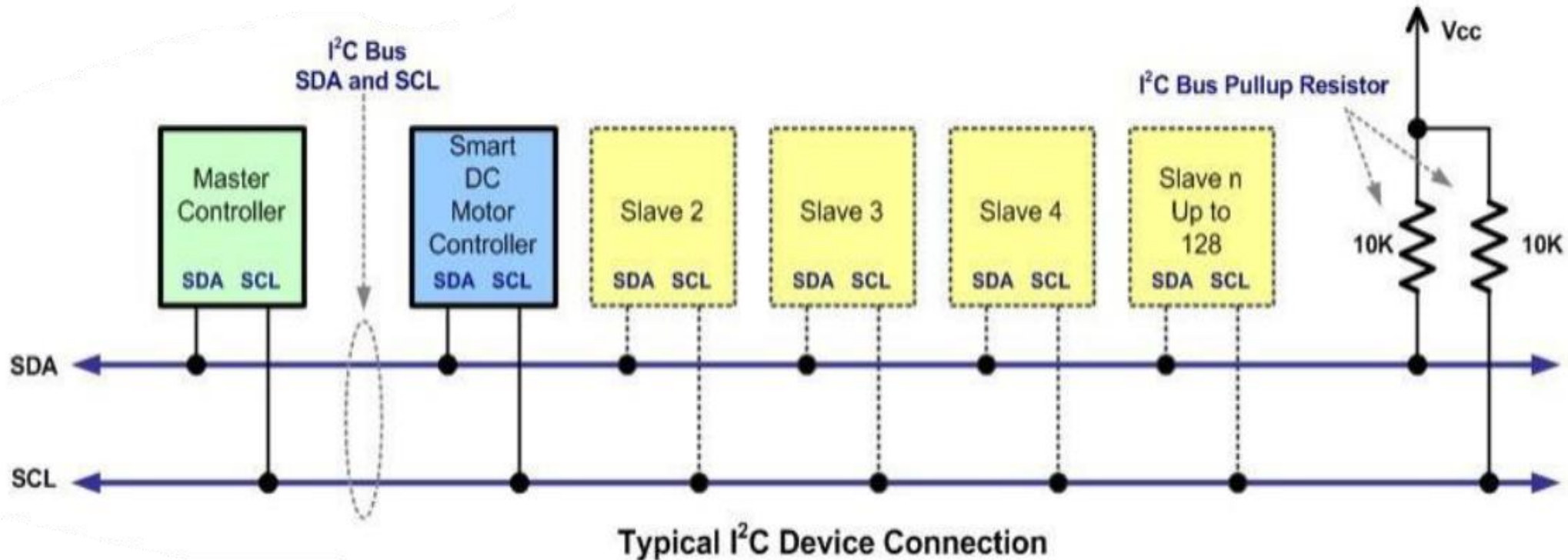
I2C-Inter - Integrated Circuit



Figure 7: Single master and slave in the I2C protocol

I2C-Inter - Integrated Circuit

Device Connection



Two Lines:

- **SDA (Serial Data Line)** → Transfers data.
- **SCL (Serial Clock Line)** → Synchronizes data transfer.

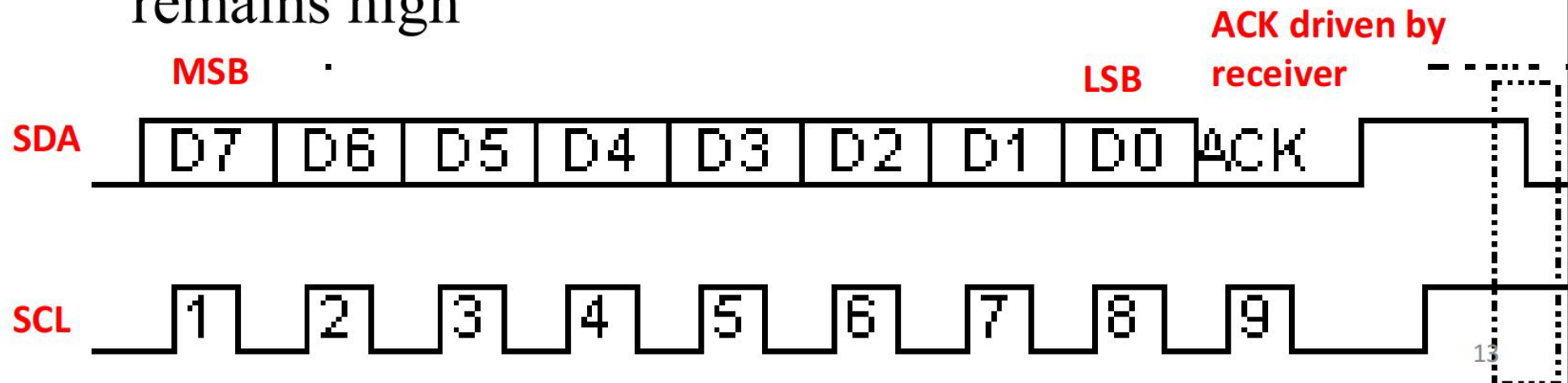
Pull-up resistors are required on SDA & SCL to maintain HIGH levels.

Supports **multiple slaves** using **unique 7-bit or 10-bit addresses**

I2C Bus operation

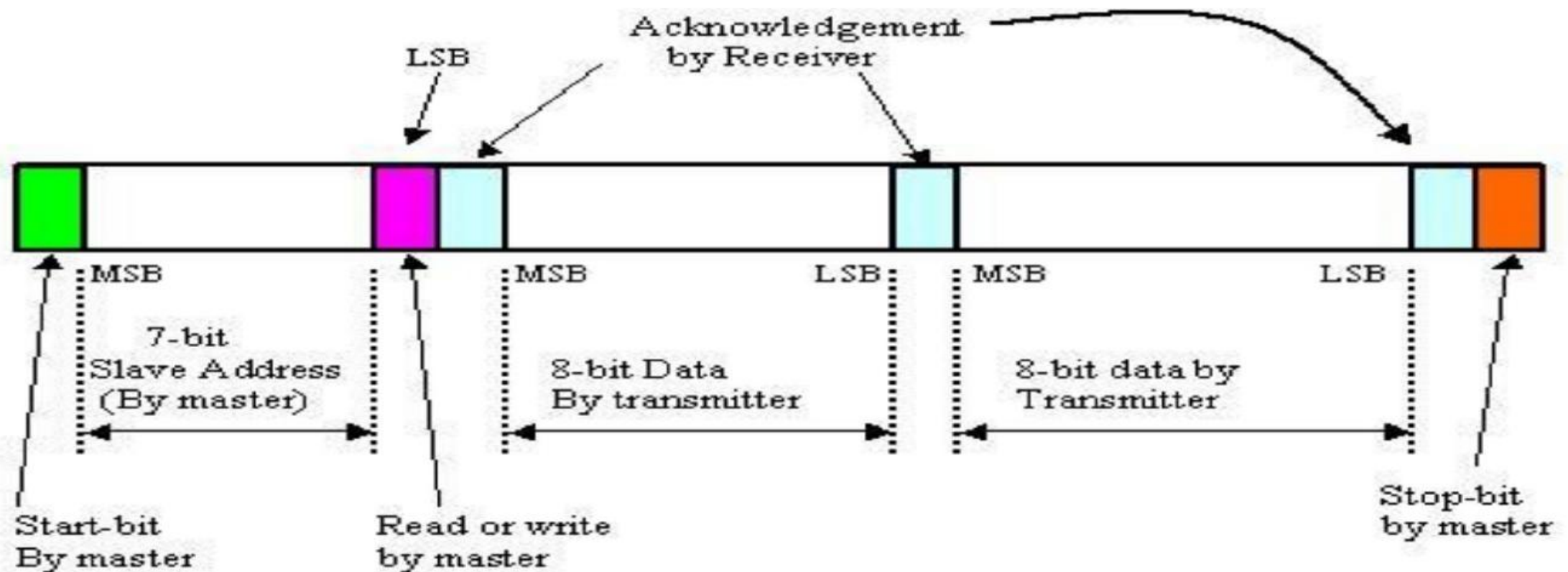


- SDA line Transmits/Receives data bits (**MSB is sent first**)
- Data in SDA line is stable during clock (**SCL**) high
- **Serial clock is driven by the master**
- **Acknowledgment bit** is driven by the **receiver** after the end of reception
- If the receiver does not acknowledge, SDA line remains high



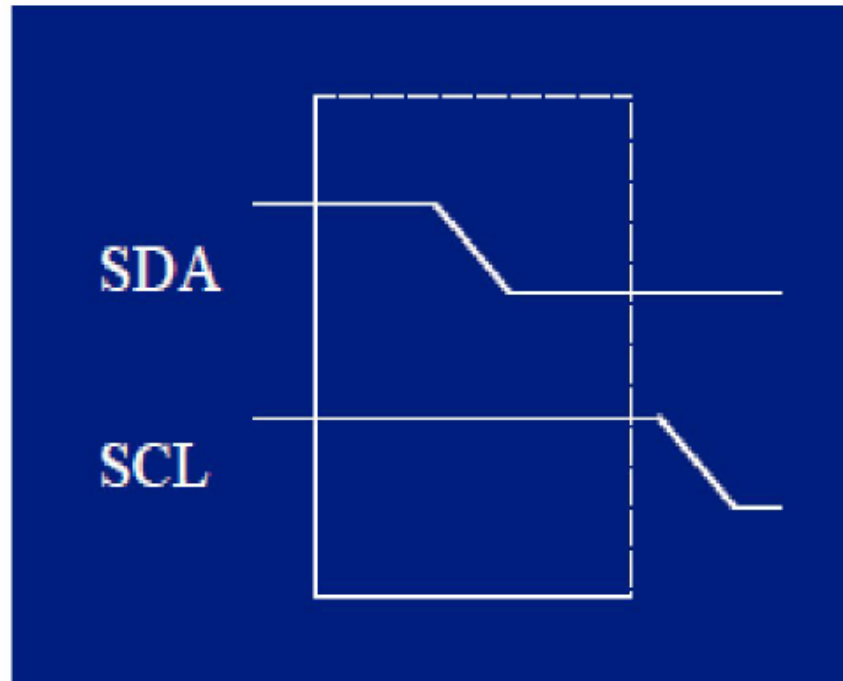
I2C Bus operation

- The format of I2C bus transfers



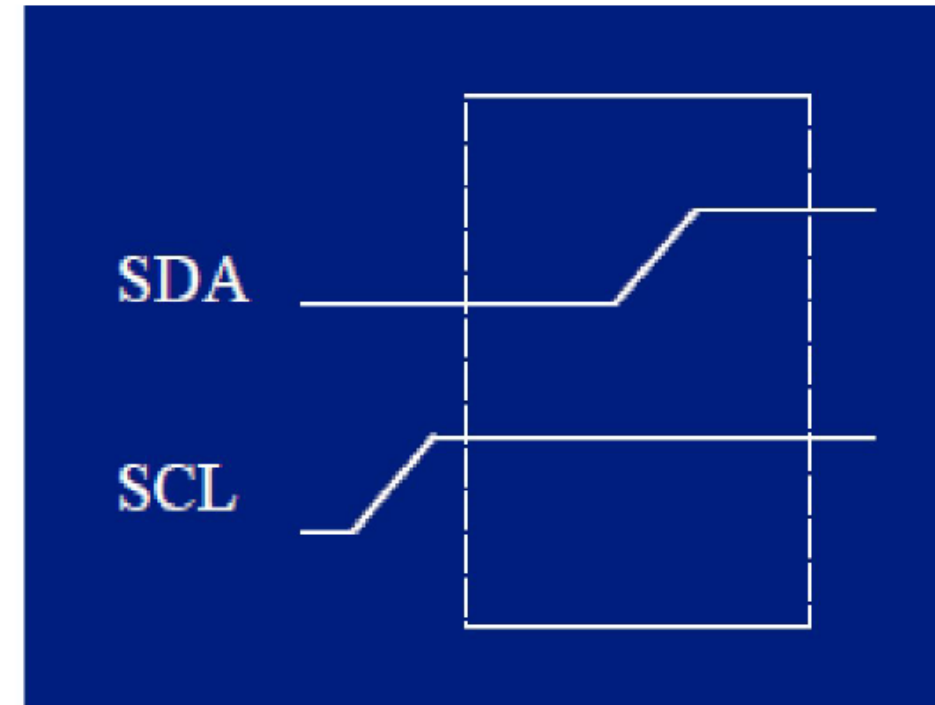
Start Bit

- Initializes I2C Bus
- SDA is pulled low, then SCL is pulled low

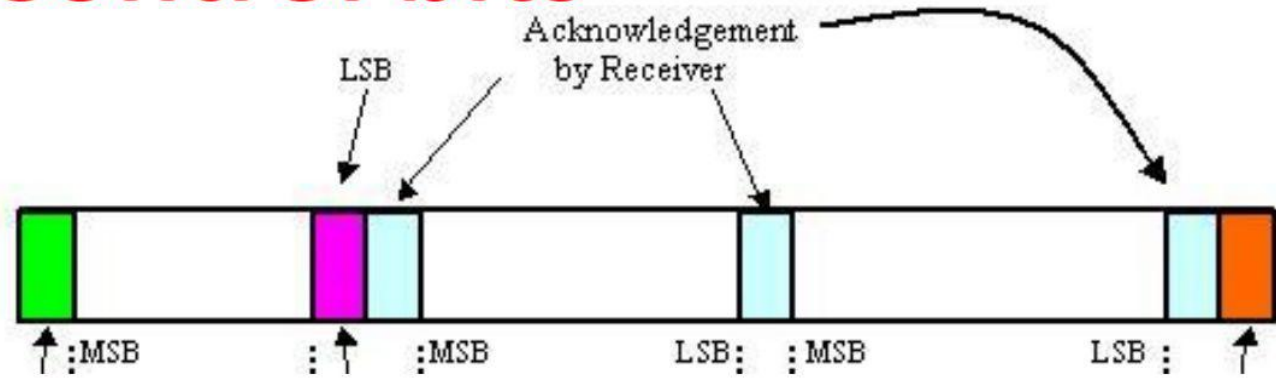


Stop Bit

- Releases I2C Bus
- SCL is released first, then SDA is released



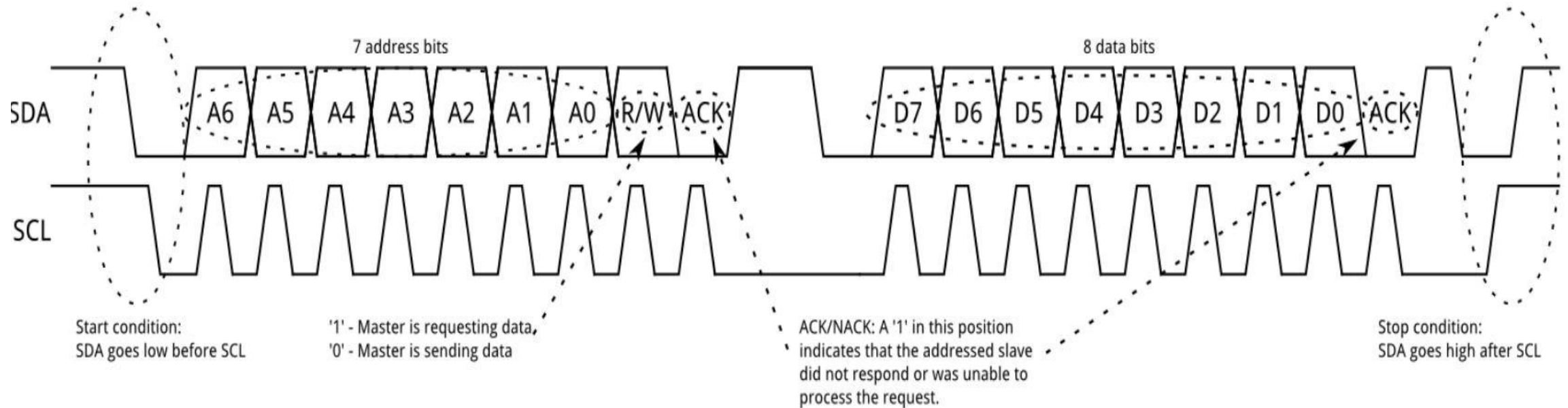
Control bits



- The start bit is followed 7 bit slave address
- Address bit is followed by R/W bit
- If $R/W=0$, subsequent bytes transmitted on the bus will be **written by the controller** to the selected peripheral
- If $R/W=1$, subsequent bytes will be sent by the selected peripheral and **read by the controller**

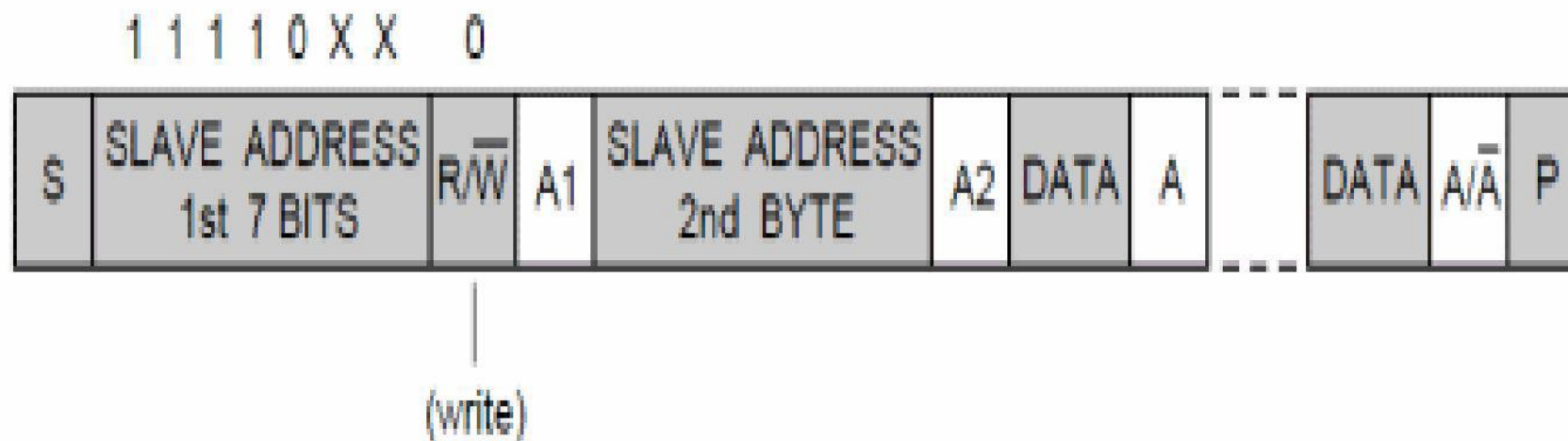


I2C typical message format



I2C typical message format

- For peripheral chip that contains more than one internal register or memory address, the PIC will typically write a second byte to the chip to set a pointer to the selected internal register or the consecutive addresses that follow it

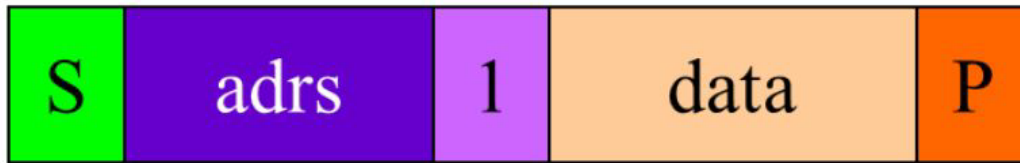


I2C typical message format

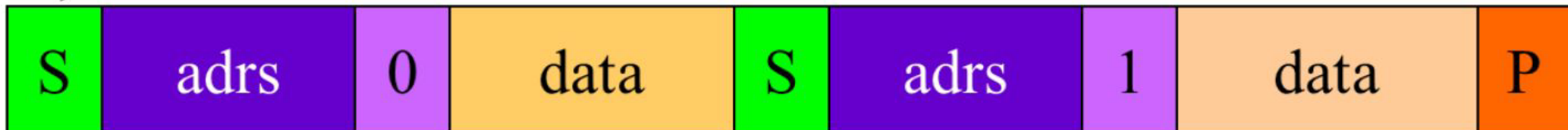
multi-byte write



read from slave



write, then read



Advantages of I2C protocol

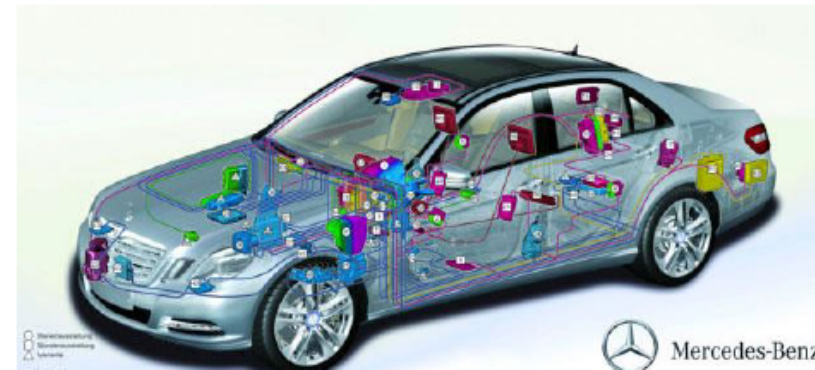
1. Simple and effective
2. It occupies less space as it directly connects the components and reduces the cost of interconnection
3. It supports multi-mastering, i.e., any device that can send or receive data can be a master
4. The ACK/NACK feature supports a robust error handling mechanism

Disadvantages of I2C protocol

1. Hardware, especially software implementation, is more complicated than the SPI Half-duplex
2. Not scalable for a large number of devices
3. It takes up much more real estate on a PCB as it uses pull-up resistors

CAN Bus

- CAN is **Control Area Network**
- It was designed for automotive electronics and was first used in production cars in 1991
- A automobile uses a number of distributed embedded controllers
- The controllers provide the controls for brakes, engines, electrical power, lamps, temperature, air conditioning, car gate, front display panel, meters display panels and cruising



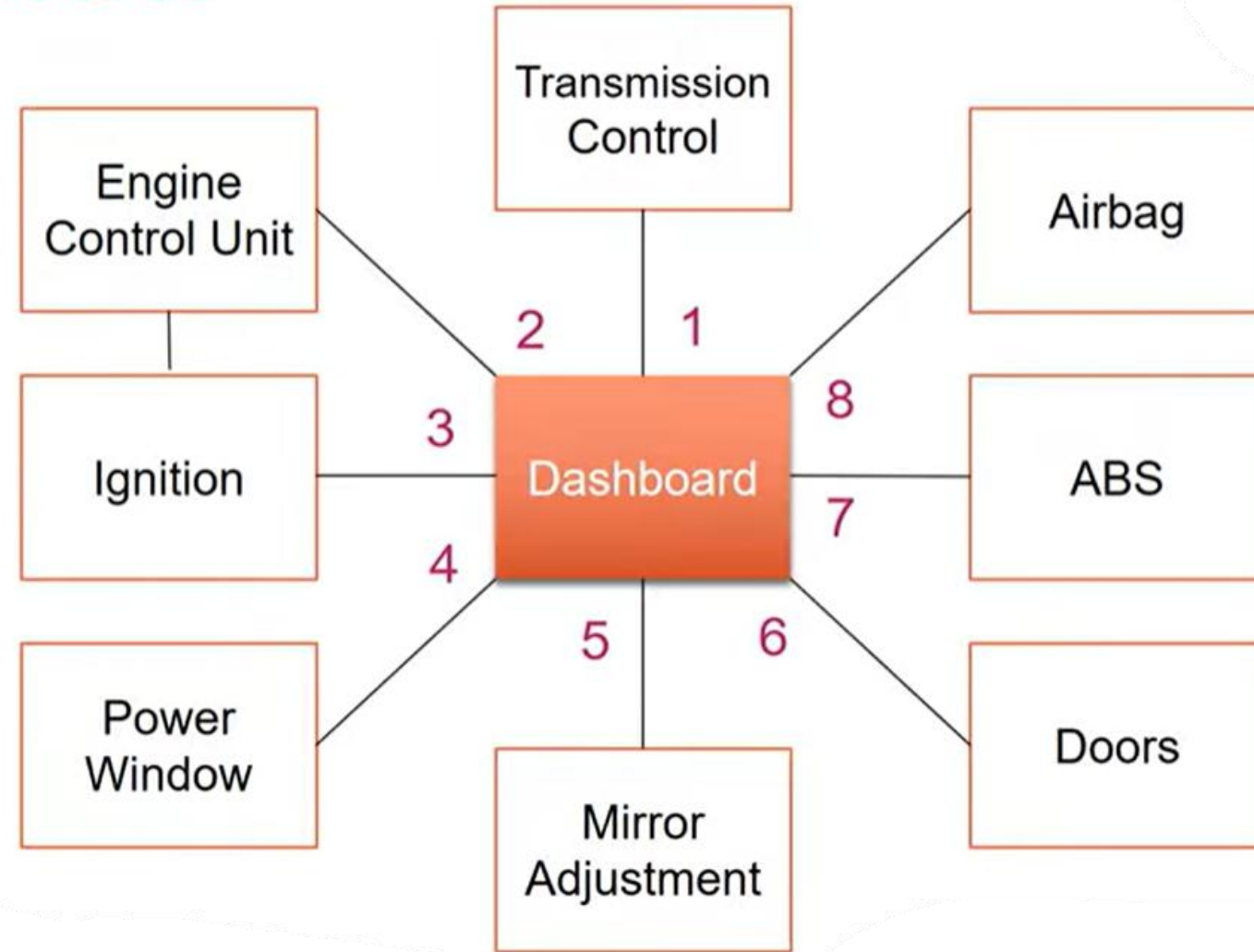
INTRODUCTION

CAN

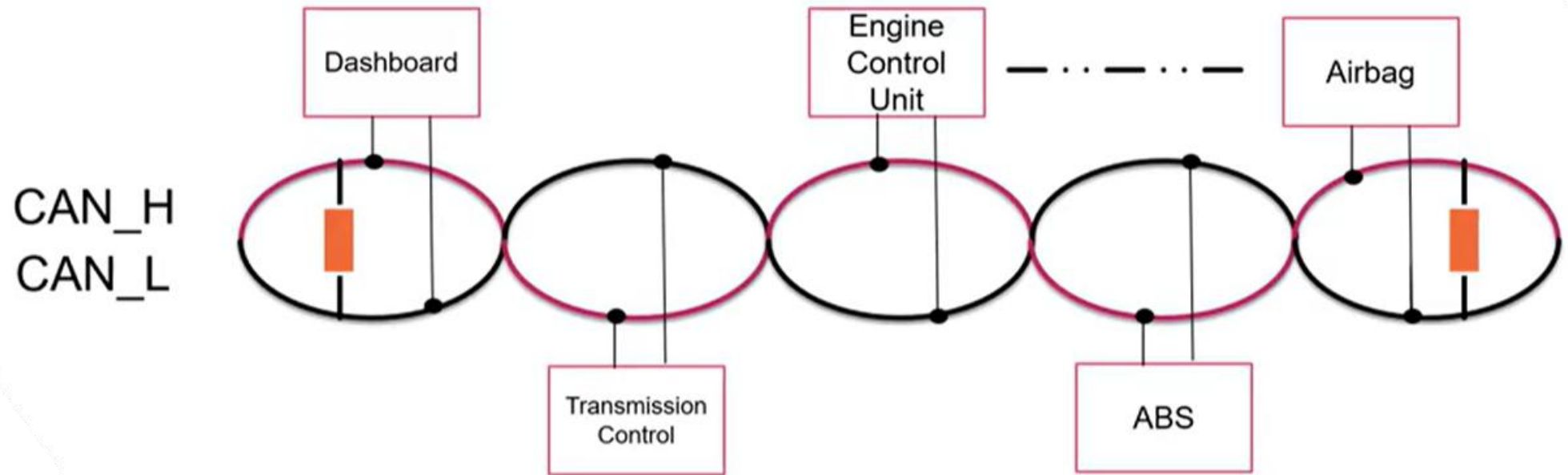
PROTOCOL

- Controller Area Network (CAN bus).
- Developed by Robert Bosch in 1986.
- No Host required.
- CAN bus is a **broadcast** type of bus.
- **Message** based protocol.
- Serial half-duplex Asynchronous Communication.
- **Differential Two** wired communication.
- Designed originally for automobiles, but also used in many other contexts.
- One of five protocols used in the on-board diagnostics.

WHY CAN?



WHY CAN?



- Low Cost
- Efficient

- Flexible
- Centralized

- Robust

APPLICATIONS CAN PROTOCOL

- Automotive (Passenger vehicles, trucks, buses)
- Electronic equipment for aviation and navigation.
- Industrial automation and mechanical Control.
- Elevators & escalators.
- Building automation.
- Medical instruments and equipment.
- Marine, Military, Industrial, Medical

LAYERED ARCHITECTURE

Reference to OSI model :-

7

Application

6

Presentation

5

Session

4

Transport

3

Network

2

Data Link

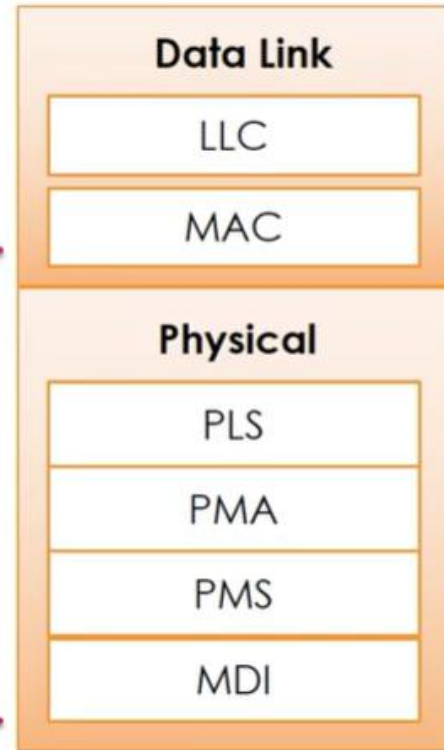
1

Physical



ISO/OSI model

CAN Layered Architecture :-



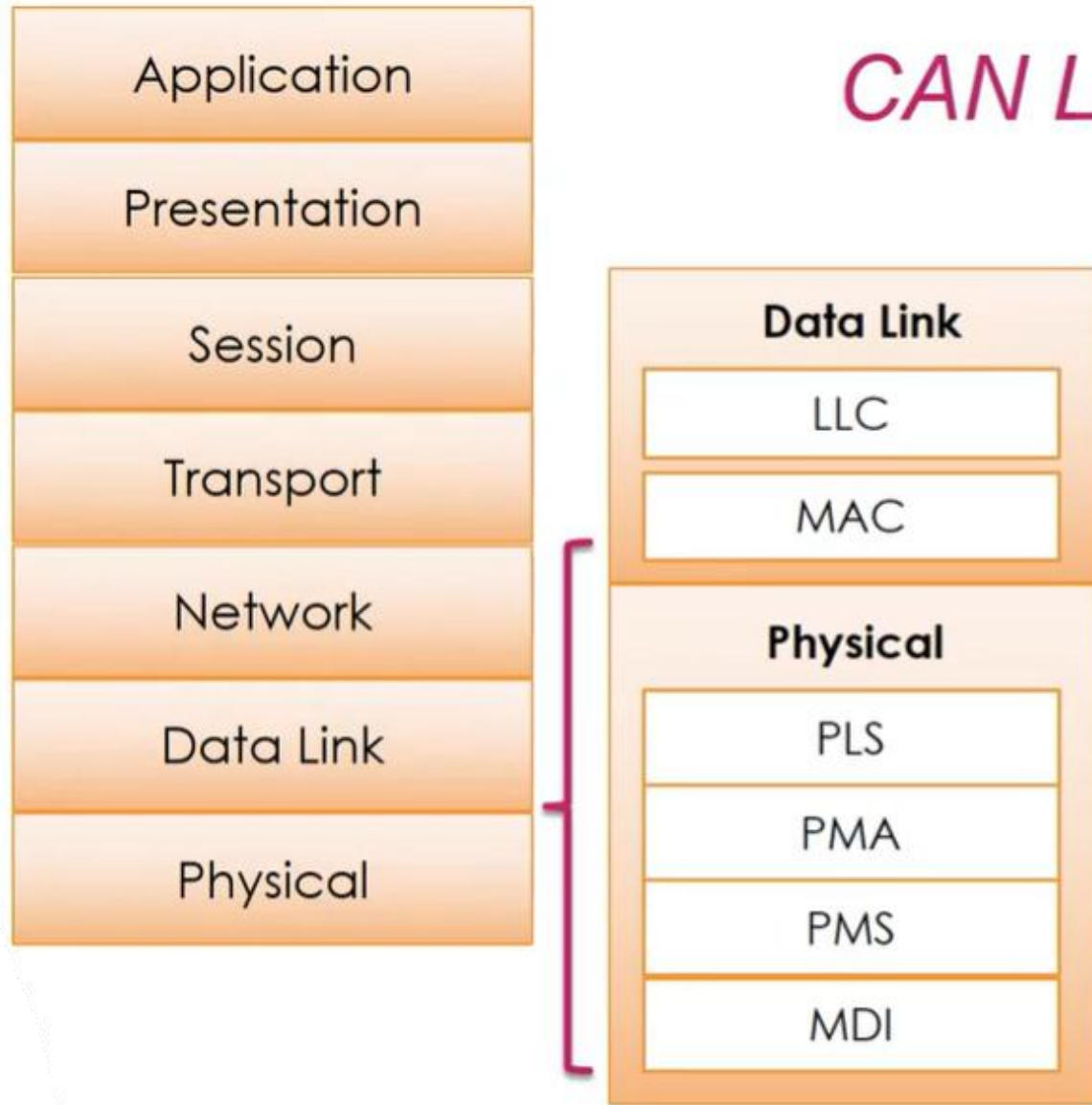
MAC : Medium Access Control

- Medium Access
- Encapsulation/Decapsulation
- Error Detection
- Signalling

LLC : Logical Link Control

- Frame Acceptance Filtering
- Overload Notification
- Recovery Management

CAN Layered Architecture :-



LLC : Logical Link Control

MAC : Medium Access Control

PLS : Physical Layer Signalling

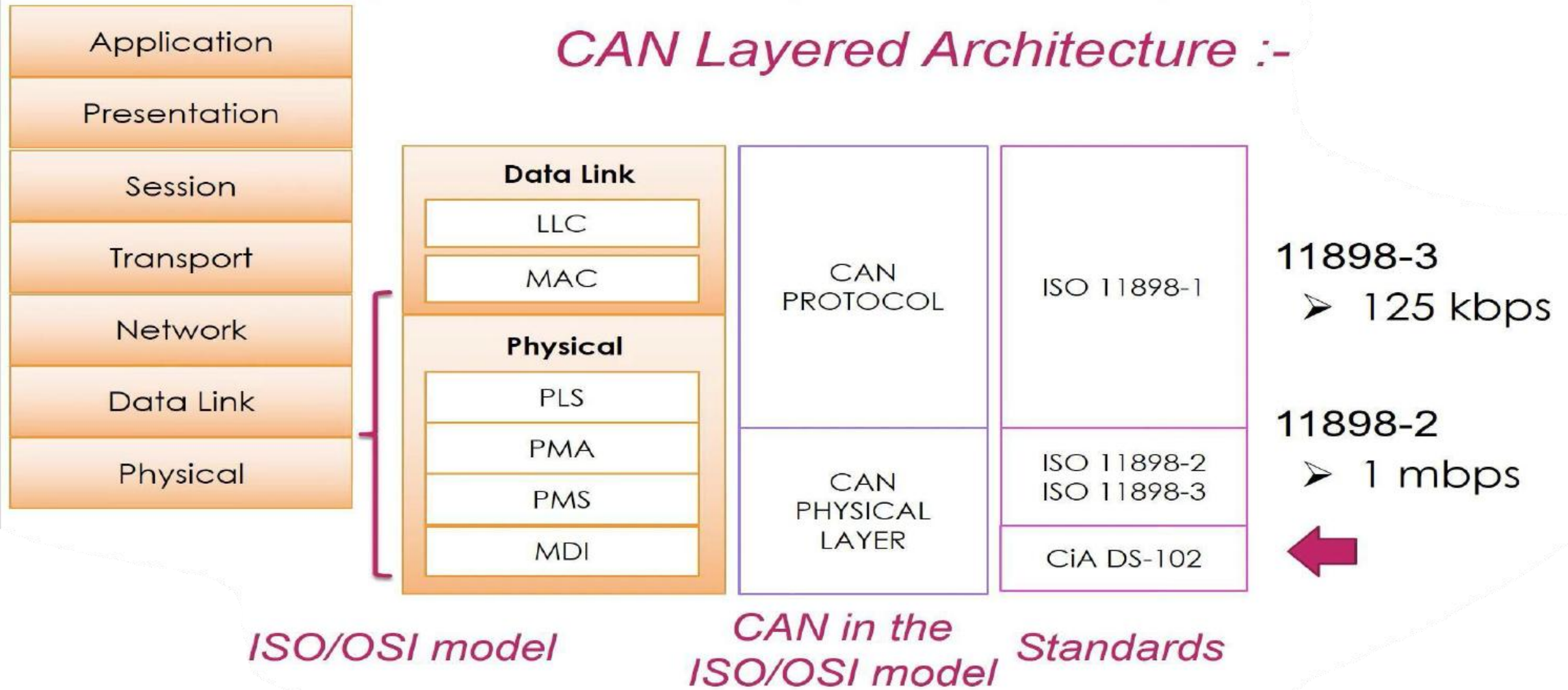
PMA : Physical Medium Attachment

PMS : Physical Medium Specification

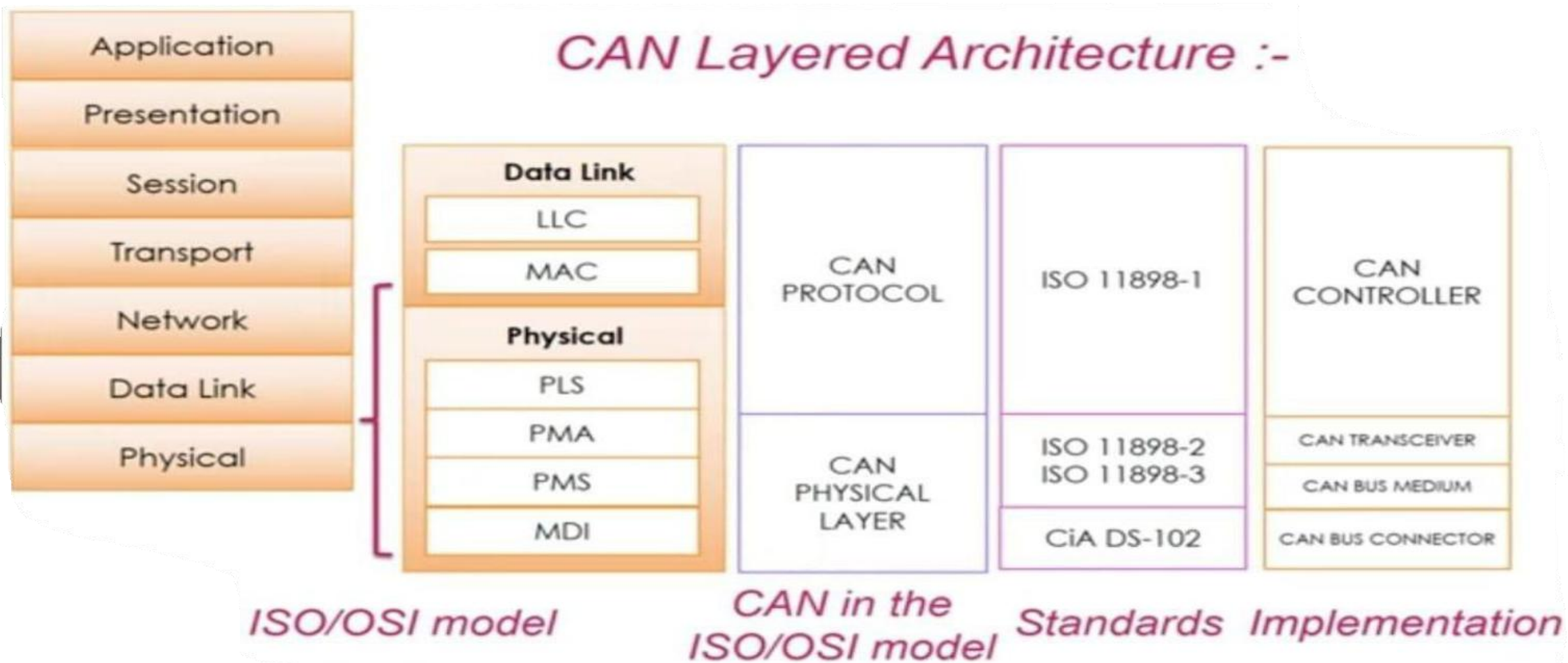
MDI : Medium Dependent Interface

ISO/OSI model

CAN Layered Architecture :-

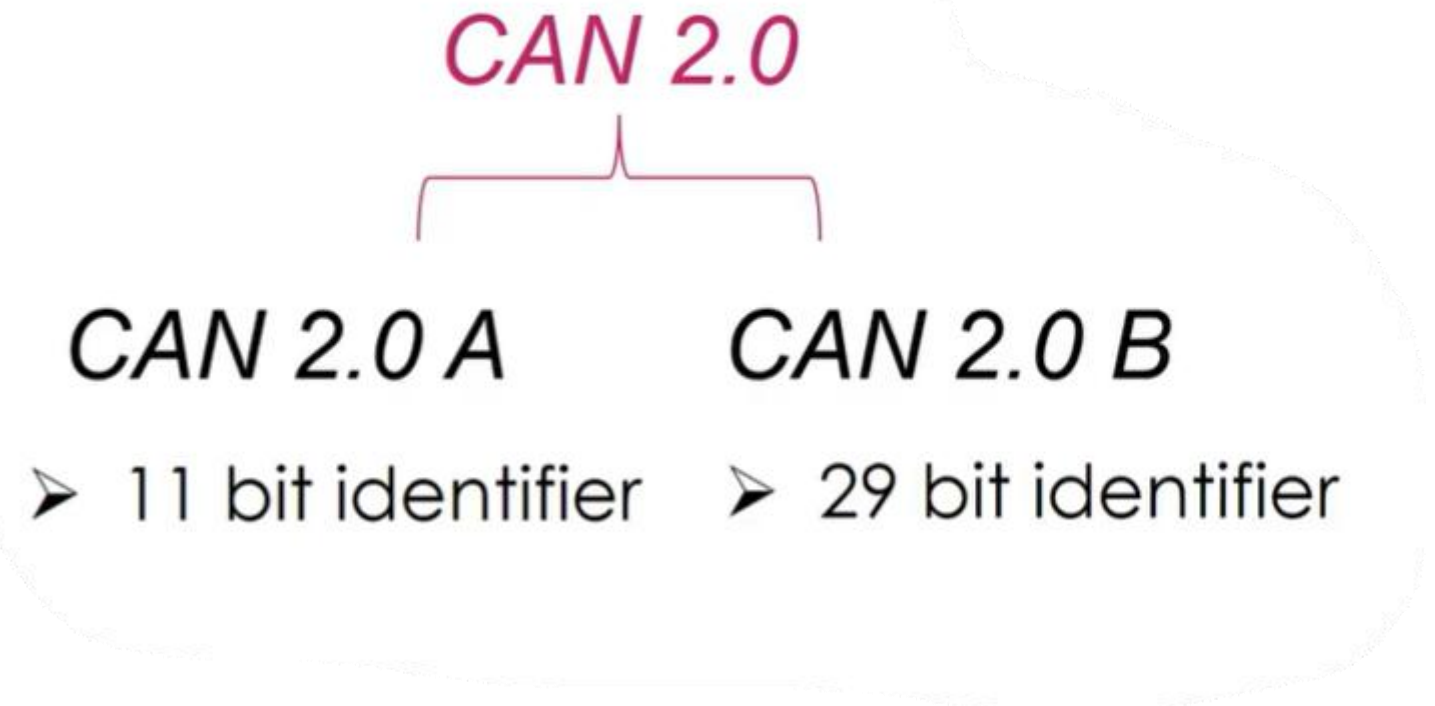


CAN Layered Architecture :-



CAN VERSIONS

CAN 2.0



CAN 2.0 A

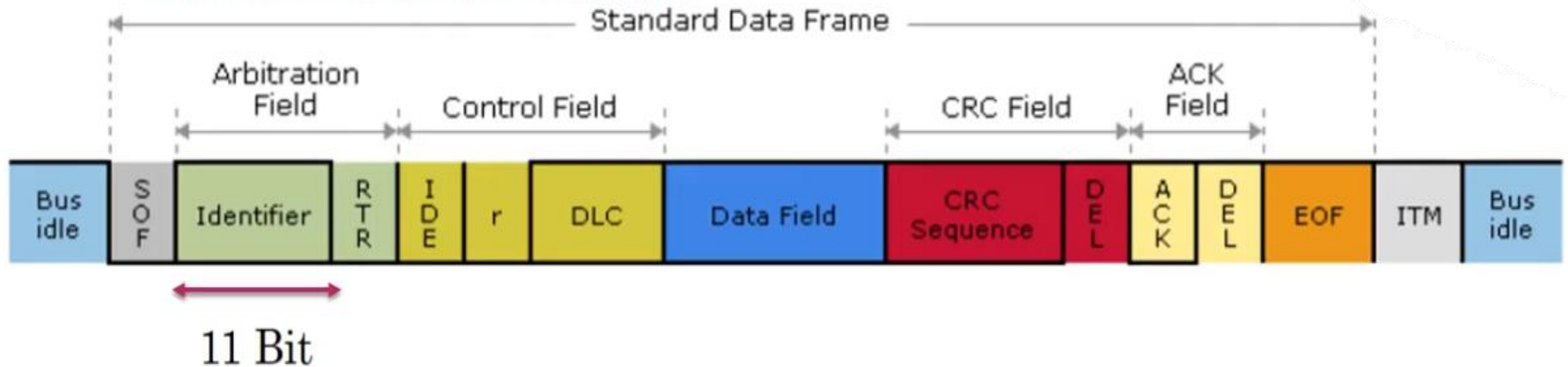
CAN 2.0 B

➤ 11 bit identifier

➤ 29 bit identifier

CAN Framing

- Standard Data Frame



- SOF – Start of Frame (1 Bit)
- Identifier – A message identifier sets the priority of the data frame.
- RTR – Remote Transmission Request, defines the frame type (data frame or remote frame) (1 Bit).

IDE (Identifier Extension) – 1 bit

- 0 → Standard frame (11-bit)
- 1 → Extended frame (29-bit)

r (Reserved bit)

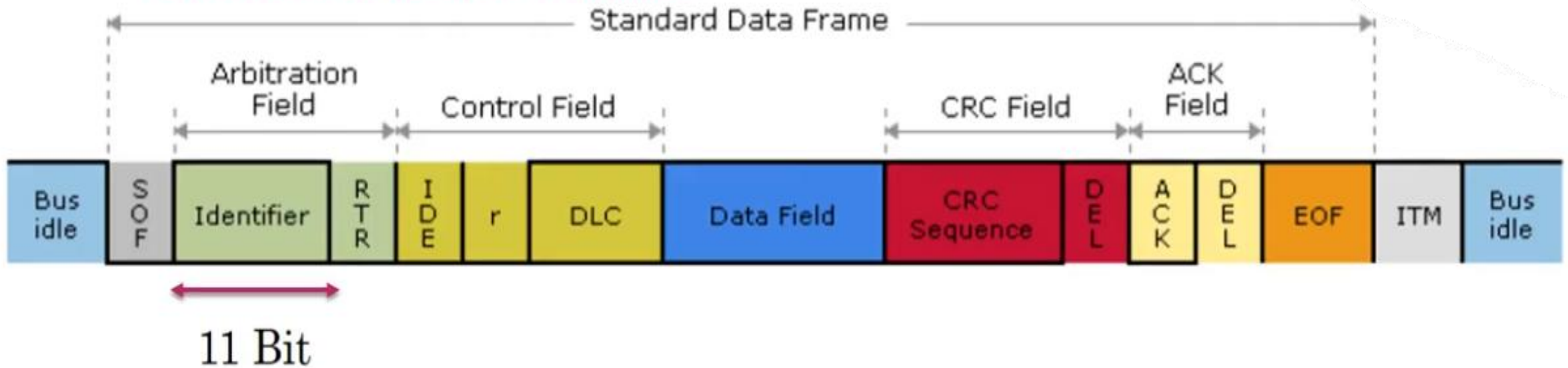
- Reserved for future use
- Always 0

DLC (Data Length Code) – 4 bits

- Specifies **data size**
- Range: **0 to 8 bytes**

CAN Framing

- Standard Data Frame



Data Field

- Actual data, 0 to 8 bytes
Example: speed, temperature

CRC Field

- **CRC Sequence**
- Error detection bits

CRC DEL (Delimiter)

- Marks end of CRC
- Always **1 (recessive)**

ACK Slot

Receiver sends:

- 0 → Data received correctly
- 1 → No acknowledgement

ACK DEL

Separator, Always **1**

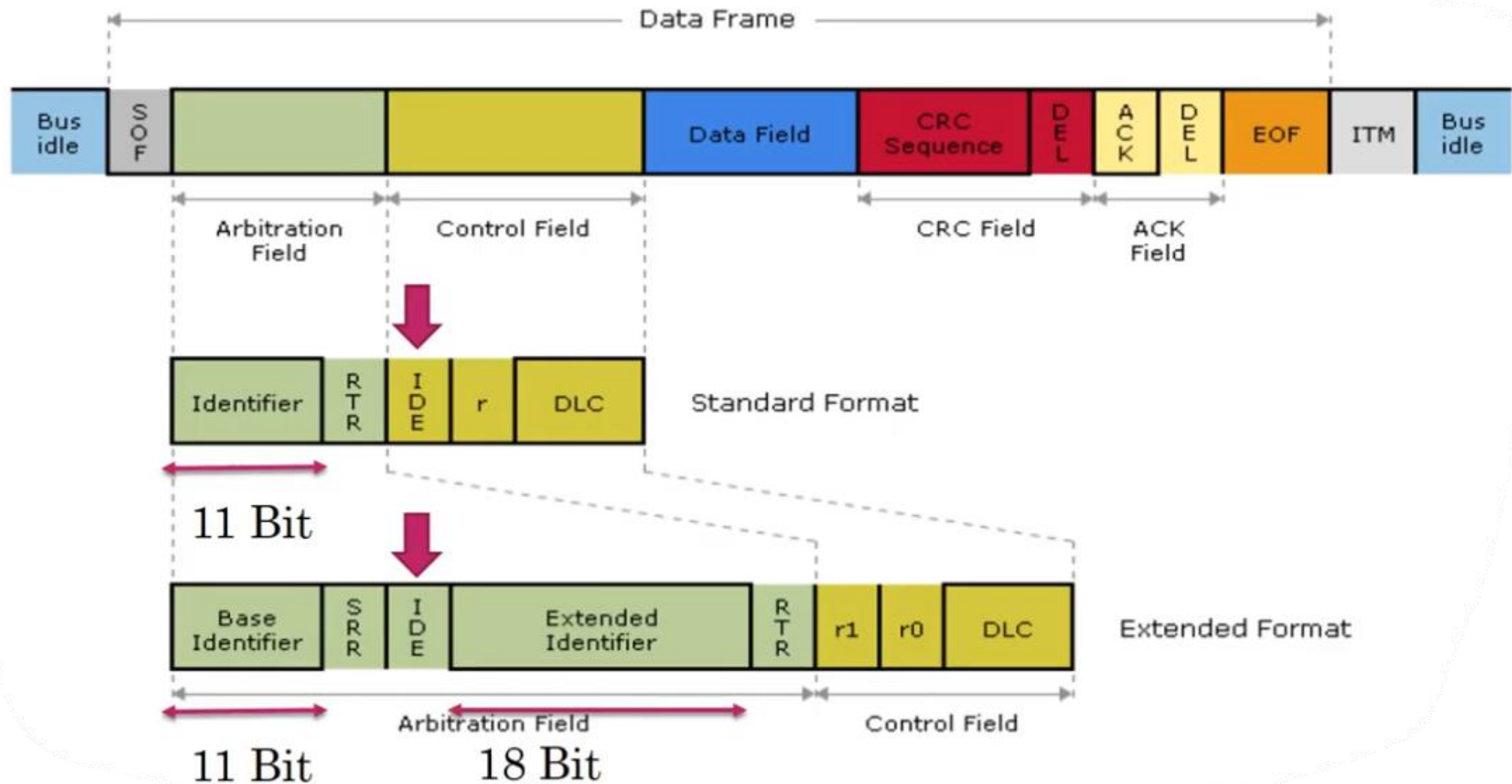
EOF (End of Frame) – 7 bits

- Marks end of frame
- All bits = 1

ITM (Intermission)

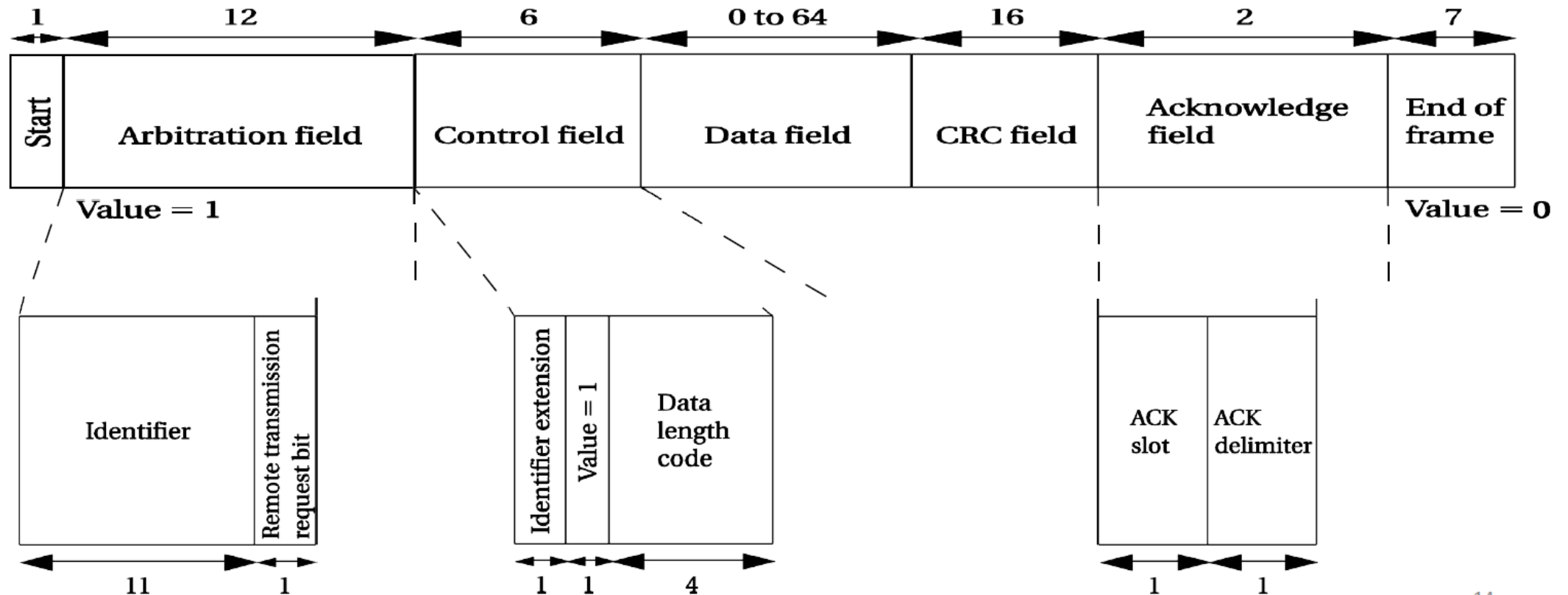
- Gap between frames
- Minimum 3 bits

CAN Framing



CAN data frame format

- A data frame **starts with a 1** and **ends with a string of seven zeroes**.

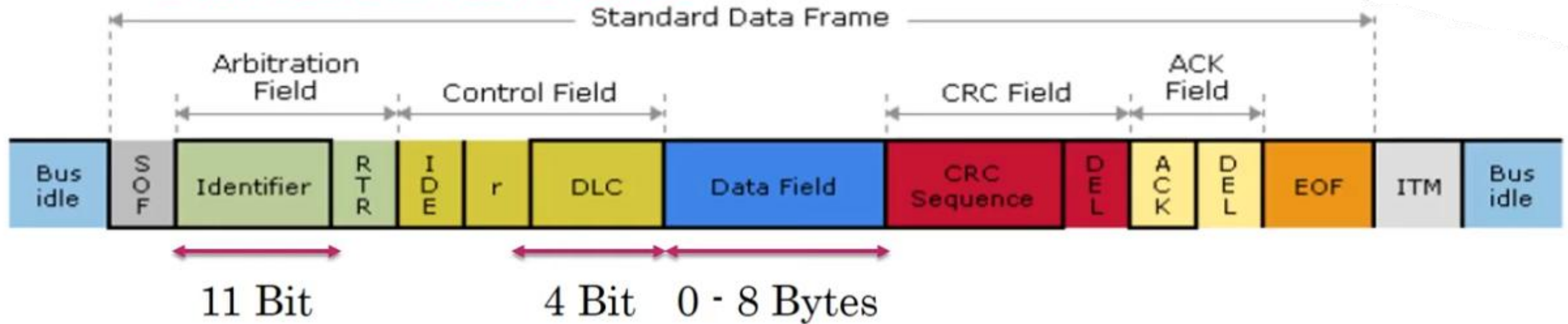


CAN Protocol defined frame bits

- **First field of 12 bits — arbitration field.**
 - 11-bit destination address and RTR bit (Remote Transmission Request)
 - Destination device address specified in an 11-bit sub- field and whether the data byte being sent is a data for the device or a request to the device in 1-bit sub-field.
 - Maximum 2^{11} devices can connect a CAN controller in case of 11-bit address field standard
 - If RTR $\square 1$, the packet is for the device at destination address
 - If RTR $\square 0$, the packet is a request for the data from the device

CAN Framing

- Standard Data Frame

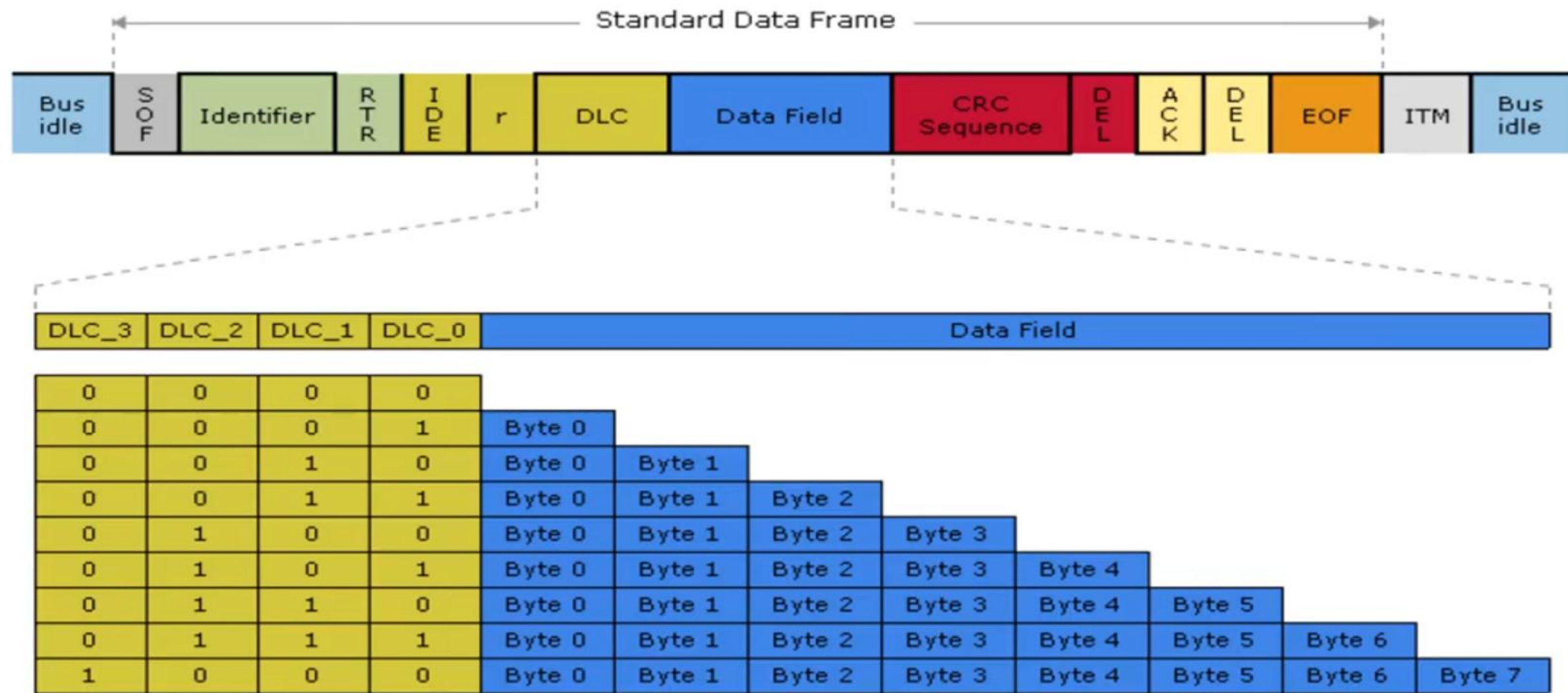


- SOF – Start of Frame (1 Bit)
- Identifier – A message identifier sets the priority of the data frame.
- RTR – Remote Transmission Request, defines the frame type (data frame or remote frame) (1 Bit).
- Control Field – User defined Functions.
- DLC – Data Length Code (4 Bits).
- Data Field – User defined Data (0 to 8 Bytes)

CAN Protocol defined frame bits

- **Second field of 6 bits— control field.**
 - The first bit is for the identifier's extension.
 - The second bit is always '1'.
 - The last 4 bits specify code for data length

CAN Framing



CAN Protocol defined frame bits

- **Fourth field of 16 bits CRC (Cyclic Redundancy Check) bits.**
 - The receiver node uses it to detect the errors, if any, during the transmission

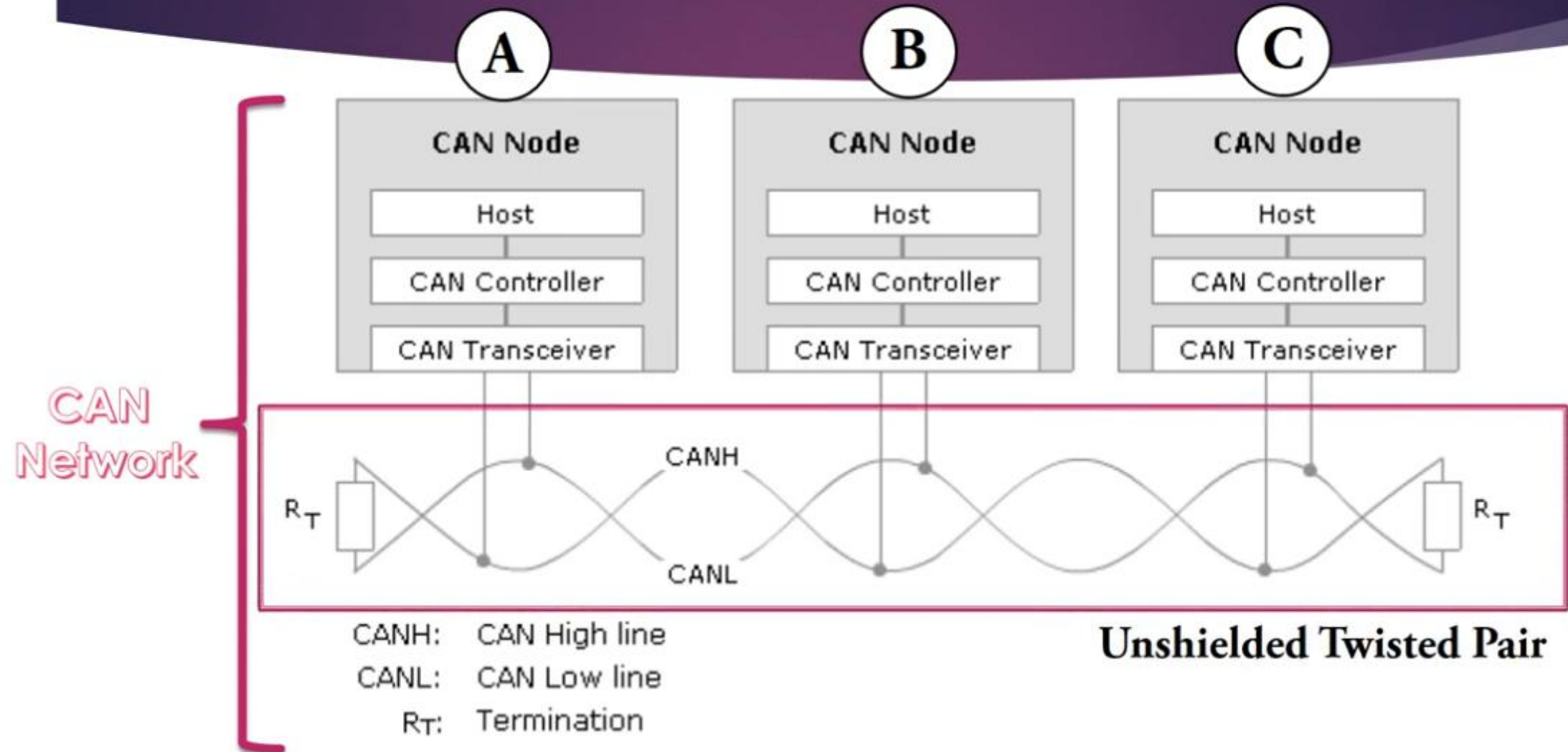
CAN Protocol defined frame bits

- **Fifth field of 2 bits**
- First bit **'ACK slot'**
 - ACK = '1' and receiver sends back '0' in this slot when the receiver detects an error in the reception. Sender after sensing '0' in the ACK slot, generally retransmits the data frame.
- Second bit **'Ack delimiter'**
 - Second bit 'ACK delimiter' bit. It signals the end of ACK field. If the transmitting node does not receive any acknowledgement of data frame within a specified time slot, it should retransmit.

CAN Protocol defined frame bits

- **Sixth field of 7-bits**
 - end- of-the frame specification and has seven '0's

CAN ELECTRICAL CHARACTERISTICS :-

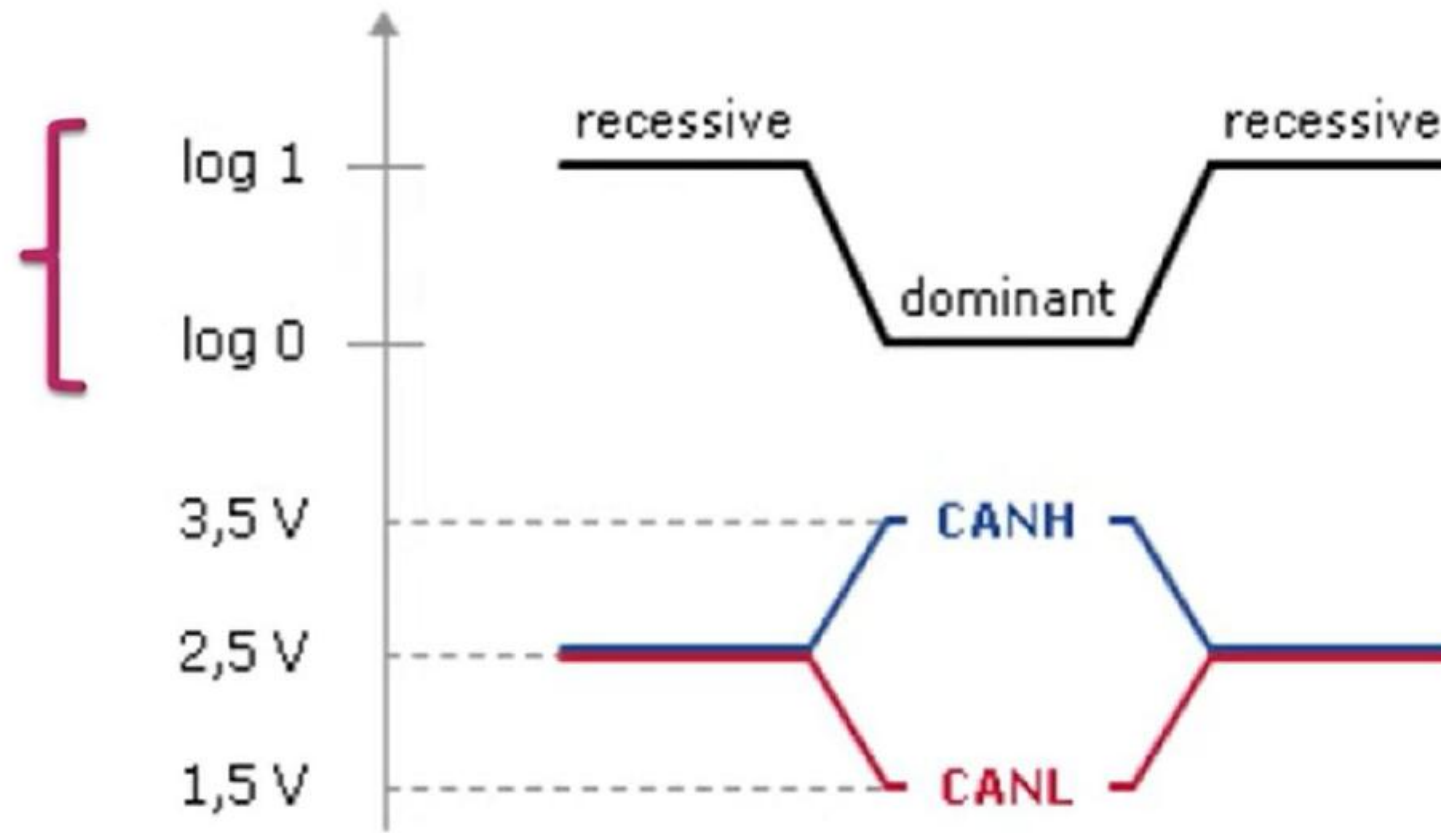


Terms associated with CAN protocol.

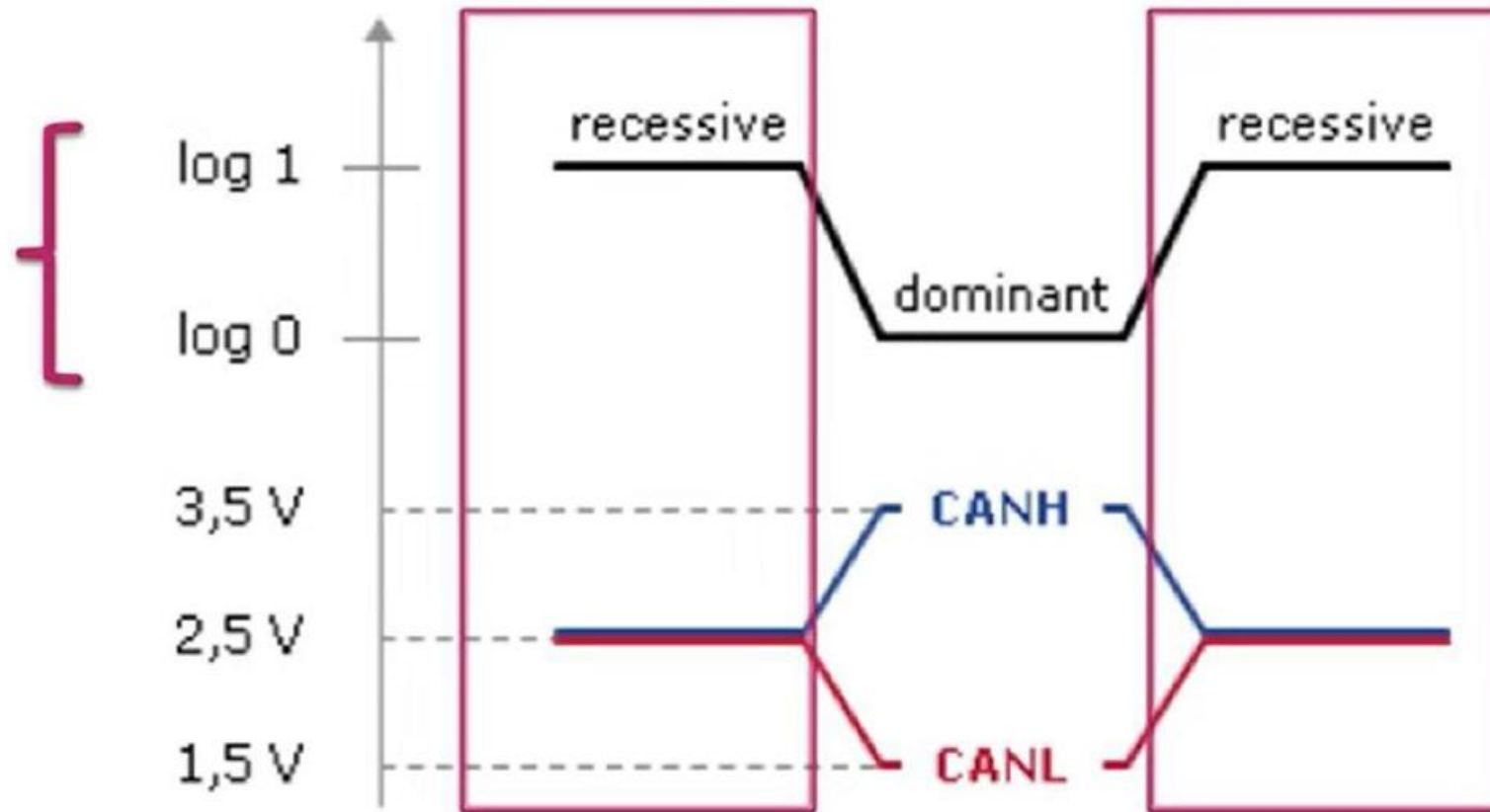
- **BUS VALUES**
- Binary values in CAN protocol are termed as **dominant and recessive bits**.
 - CAN define the **logic “0”** as **dominant bit**.
 - CAN define the **logic “1”** as **recessive bit**.
- In the CAN system dominant bit always overwrites the recessive bit.

HOW ARBITRATION WORKS IN CAN PROTOCOL

CAN LOGIC



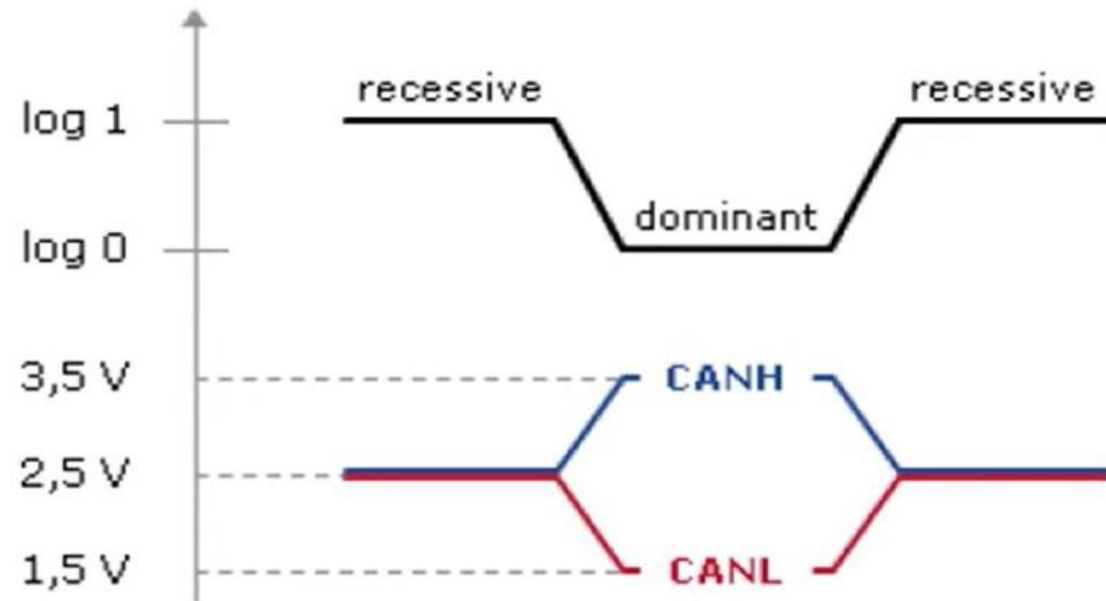
HOW ARBITRATION WORKS IN CAN PROTOCOL



HOW ARBITRATION WORKS IN CAN PROTOCOL

CAN ELECTRICAL CHARACTERISTICS :-

- ★ Logic 1 or Recessive
CANH & CANL both are at 2.5V
- ★ Logic 0 or Dominant
CANH at 3.5V & CANL at 1.5V
- ★ Idle bus state is Recessive
- ★ Dominant state dominates Recessive state



HOW ARBITRATION WORKS IN CAN PROTOCOL

CAN BUS LOGIC :-

Wired-AND Logic

Sender 1	0	1	0	1	0	1	0	1
Sender 2	0	0	1	1	0	0	1	1
Sender 3	0	0	0	0	1	1	1	1
CAN Bus	0	0	0	0	0	0	0	1

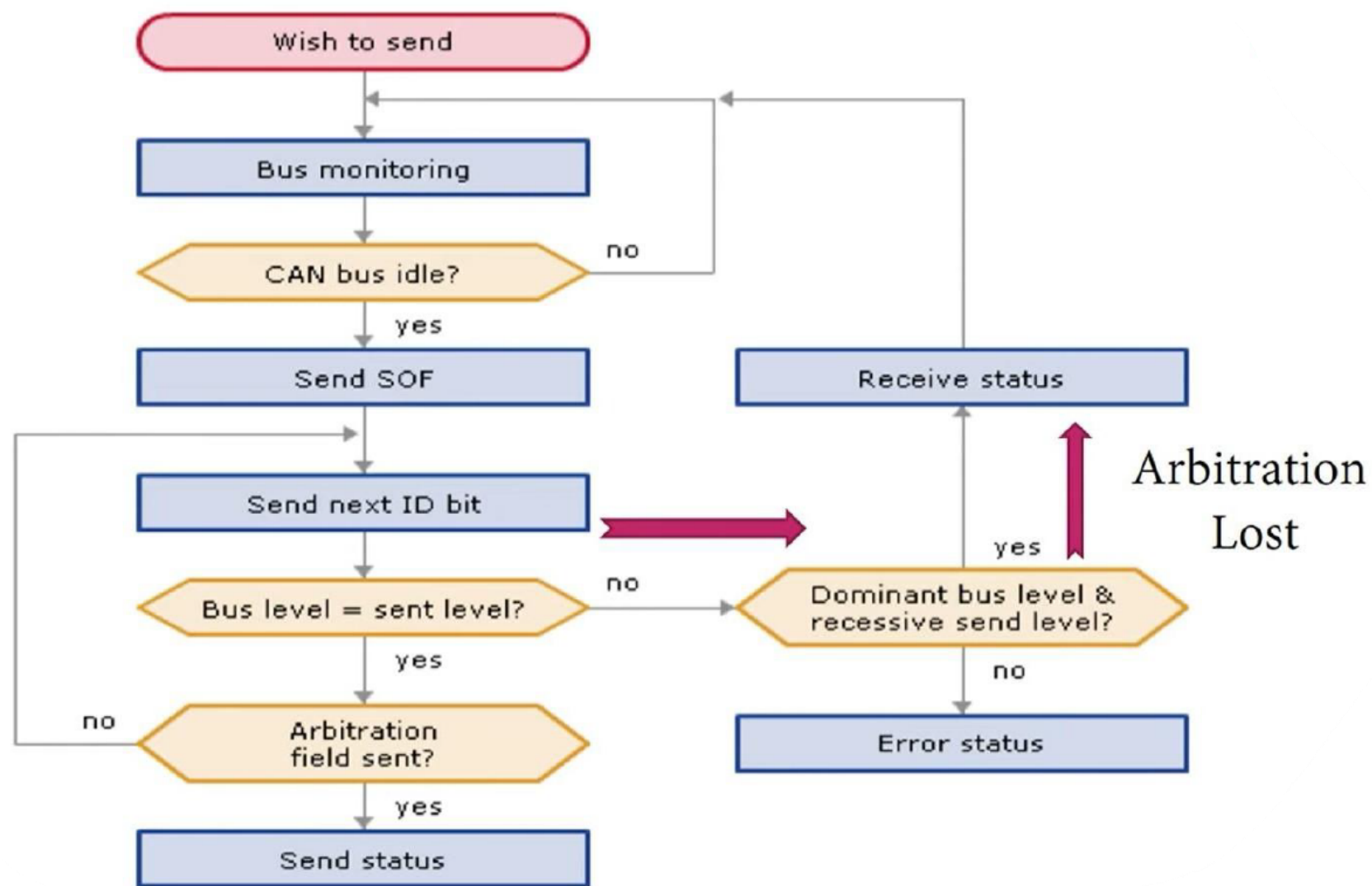
Dominant

Wired-AND Logic

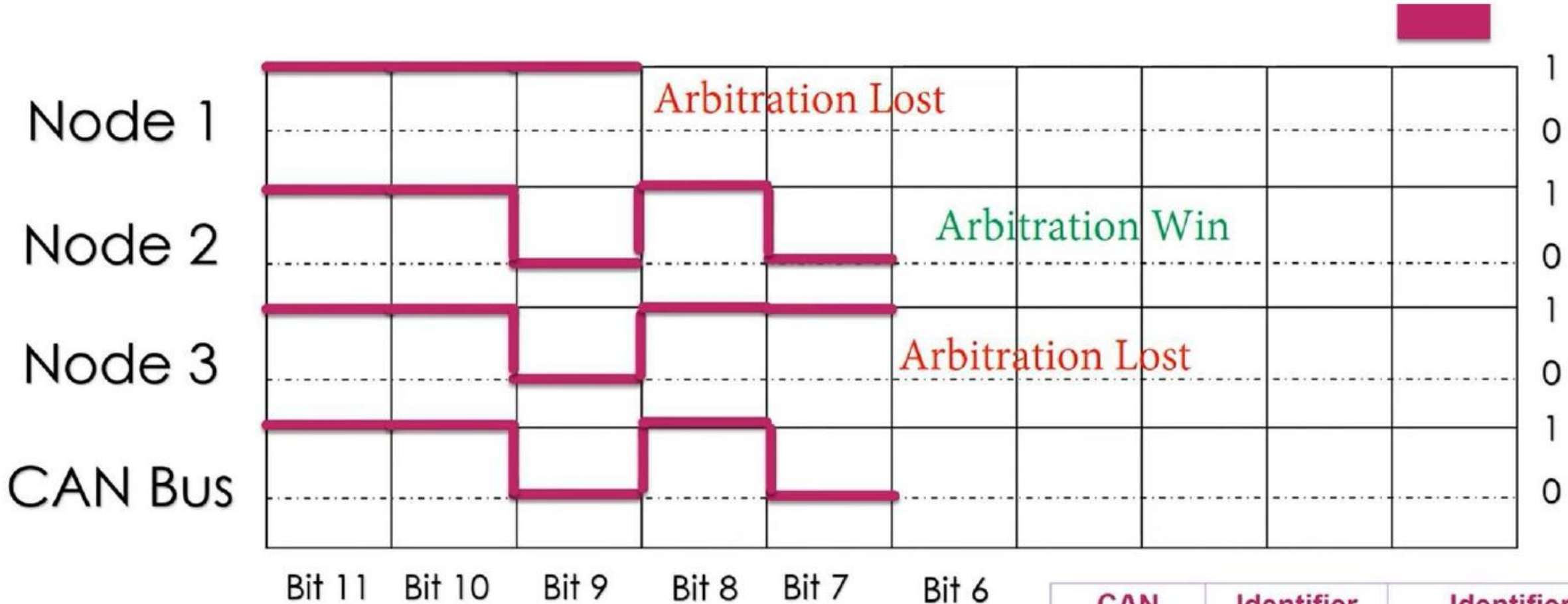
Sender 1	0	1	0	1	0	1	0	1
Sender 2	0	0	1	1	0	0	1	1
Sender 3	0	0	0	0	1	1	1	1
CAN Bus	0	0	0	0	0	0	0	1

Recessive

CAN COMMUNICATION PRINCIPLE



CAN ARBITRATION EXAMPLE



CAN Node	Identifier (Hex)	Identifier (Binary)
1	0x7F3	11111110011
2	0x6B3	11010110011
3	0x6D9	11011011001